唐诗意象智能分析系统

— 源代码正文 —

（共计 31 个源文件，6225 行）

文件：config.py （112 行，5.4 KB）

```
# -*- coding: utf-8 -*-
"""
唐诗意象智能分析系统 — 全局配置
"""
import os
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
# ═══════════════════════════════════════════════════════════════
# API 配置
# ═══════════════════════════════════════════════════════════════
LLM_API_BASE = "http://127.0.0.1:44787"
EMBED_API_URL = f"{LLM_API_BASE}/v1/embeddings"
CHAT_API_URL = f"{LLM_API_BASE}/v1/chat/completions"
EMBED_MODEL = "text-embedding-qwen3-embedding-4b"
CHAT_MODEL = "qwen3.5-35b-a3b"
# ═══════════════════════════════════════════════════════════════
# 路径配置
# ═══════════════════════════════════════════════════════════════
POEMS_JSON_DIR = os.path.join(BASE_DIR, "json_dataa")
POEM_JSON_DIR = os.path.join(BASE_DIR, "poem_json")
RAG_DB_DIR = os.path.join(BASE_DIR, "rag_db")
TEMPLATES_DIR = os.path.join(BASE_DIR, "templates")
EXPORT_DIR = os.path.join(BASE_DIR, "exports")
LOG_DIR = os.path.join(BASE_DIR, "logs")
CACHE_DIR = os.path.join(BASE_DIR, "cache")
TESTS_DIR = os.path.join(BASE_DIR, "tests")
# ═══════════════════════════════════════════════════════════════
# RAG 检索配置
# ═══════════════════════════════════════════════════════════════
TOP_K = 5
RAG_COLLECTION_NAME = "poems"
RAG_SPACE_FUNC = "cosine"
# ═══════════════════════════════════════════════════════════════
# Flask 运行配置
# ═══════════════════════════════════════════════════════════════
FLASK_HOST = "0.0.0.0"
FLASK_PORT = 5000
FLASK_DEBUG = True
FLASK_THREADED = True
SECRET_KEY = os.environ.get("FLASK_SECRET_KEY", "tang-poetry-secret-key-change-in-production")
# ═══════════════════════════════════════════════════════════════
# 数据分页
# ═══════════════════════════════════════════════════════════════
PAGE_SIZE = 25
RENDER_LIMIT = 5000
# ═══════════════════════════════════════════════════════════════
# 文件处理
# ═══════════════════════════════════════════════════════════════
MAX_BYTES_PER_CHUNK = 1024
EMBEDDING_RETRIES = 3
EMBEDDING_TIMEOUT = 30
CHAT_TIMEOUT = 300
# ═══════════════════════════════════════════════════════════════
# 意象分类映射
# ═══════════════════════════════════════════════════════════════
CATEGORY_NAME_MAP = {
    "1-1": "自然意象-天文意象",
    "1-2": "自然意象-地理意象",
    "1-3": "自然意象-植物意象",
```

```
    "1-4": "自然意象-动物意象",
    "2-1": "社会意象-生产生活意象",
    "2-2": "社会意象-军事战争意象",
    "2-3": "社会意象-制度观念意象",
    "3-1": "人文意象-人造器物意象",
    "3-2": "人文意象-人类自身意象",
    "3-3": "人文意象-人物角色意象",
    "3-4": "人文意象-文化意象",
}
CATEGORY_MAJOR_MAP = {
    "1": "自然意象",
    "2": "社会意象",
    "3": "人文意象",
}
# ──────────────────────────────────────────────────────────────
# 已知诗列表
# ──────────────────────────────────────────────────────────────
KNOWN_AUTHORS = [
    "杜甫", "李白", "王维", "白居易", "苏轼", "李商隐", "杜牧",
    "王昌龄", "孟浩然", "柳宗元", "韩愈", "欧阳修", "辛弃疾",
    "李清照", "陶渊明", "张九龄", "司空曙", "刘长卿", "韦应物",
    "岑参", "高适", "王之涣", "元稹", "温庭筠", "晏殊",
]
# ──────────────────────────────────────────────────────────────
# 日志
# ──────────────────────────────────────────────────────────────
LOG_LEVEL = "INFO"
LOG_MAX_BYTES = 10 * 1024 * 1024  # 10MB
LOG_BACKUP_COUNT = 5
LOG_FORMAT = "%(asctime)s [%(levelname)s] %(name)s: %(message)s"
# ──────────────────────────────────────────────────────────────
# 缓存
# ──────────────────────────────────────────────────────────────
CACHE_DEFAULT_TTL = 300  # 5 分钟
CACHE_ENABLED = True
# ──────────────────────────────────────────────────────────────
# 导出
# ──────────────────────────────────────────────────────────────
EXPORT_CSV_ENCODING = "utf-8-sig"
```

-- 文件结束：config.py --

文件：logger.py (93 行, 2.3 KB)

```
# -*- coding: utf-8 -*-
"""
日志模块 — 分级日志、文件滚动、上下文跟踪
"""
import logging
import os
import sys
import traceback
from logging.handlers import RotatingFileHandler
from config import LOG_DIR, LOG_LEVEL, LOG_MAX_BYTES, LOG_BACKUP_COUNT, LOG_FORMAT
def ensure_log_dir():
    if not os.path.exists(LOG_DIR):
        os.makedirs(LOG_DIR, exist_ok=True)
def _create_console_handler(formatter):
    handler = logging.StreamHandler(sys.stdout)
    handler.setFormatter(formatter)
    handler.setLevel(logging.DEBUG)
    return handler
def _create_file_handler(log_path, formatter):
    handler = RotatingFileHandler(
        log_path,
        maxBytes=LOG_MAX_BYTES,
        backupCount=LOG_BACKUP_COUNT,
```

```
        encoding="utf-8",
    )
    handler.setFormatter(formatter)
    handler.setLevel(logging.DEBUG)
    return handler
_loggers = {}
def get_logger(name=None):
    if name is None:
        name = __name__
    if name in _loggers:
        return _loggers[name]
    ensure_log_dir()
    level = getattr(logging, LOG_LEVEL.upper(), logging.INFO)
    formatter = logging.Formatter(LOG_FORMAT)
    logger = logging.getLogger(name)
    logger.setLevel(level)
    if not logger.handlers:
        logger.addHandler(_create_console_handler(formatter))
        logger.addHandler(
            _create_file_handler(os.path.join(LOG_DIR, "app.log"), formatter)
        )
    logger.propagate = False
    _loggers[name] = logger
    return logger
class LoggerMixin:
    """混入类，为对象提供便捷日志方法"""
    @property
    def logger(self):
        if not hasattr(self, "_logger"):
            cls_name = type(self).__name__
            self._logger = get_logger(cls_name)
        return self._logger
    def log_debug(self, msg, *args, **kwargs):
        self.logger.debug(msg, *args, **kwargs)
    def log_info(self, msg, *args, **kwargs):
        self.logger.info(msg, *args, **kwargs)
    def log_warning(self, msg, *args, **kwargs):
        self.logger.warning(msg, *args, **kwargs)
    def log_error(self, msg, *args, **kwargs):
        self.logger.error(msg, *args, **kwargs)
    def log_exception(self, msg, *args, **kwargs):
        self.logger.exception(msg, *args, **kwargs)
def format_exception(e: Exception) -> str:
    return "".join(traceback.format_exception(type(e), e, e.__traceback__))
```

-- 文件结束：logger.py --

文件：errors.py (112 行, 3.0 KB)

```
# -*- coding: utf-8 -*-
"""
统一异常处理模块
"""
import json
from flask import Response, jsonify
class AppError(Exception):
    """应用基础异常"""
    def __init__(self, message: str, status_code: int = 400, details: dict = None):
        super().__init__(message)
        self.message = message
        self.status_code = status_code
        self.details = details or {}
class DataNotFoundError(AppError):
    """数据未找到"""
    def __init__(self, message="未找到相关数据", details=None):
        super().__init__(message, status_code=404, details=details)
class EmbeddingError(AppError):
```

```
    """向量化服务异常"""
    def __init__(self, message="向量化服务请求失败", details=None):
        super().__init__(message, status_code=502, details=details)
class LLMError(AppError):
    """大模型服务异常"""
    def __init__(self, message="大模型服务请求失败", details=None):
        super().__init__(message, status_code=502, details=details)
class ValidationError(AppError):
    """输入校验异常"""
    def __init__(self, message="输入参数校验失败", field=None, details=None):
        d = details or {}
        if field:
            d["field"] = field
        super().__init__(message, status_code=422, details=d)
class CacheError(AppError):
    """缓存异常"""
    def __init__(self, message="缓存操作失败", details=None):
        super().__init__(message, status_code=500, details=details)
class ExportError(AppError):
    """导出异常"""
    def __init__(self, message="数据导出失败", details=None):
        super().__init__(message, status_code=500, details=details)
def sse_error(message: str) -> str:
    """生成 SSE 格式错误消息"""
    return f"data: {json.dumps({'type': 'error', 'message': message}, ensure_ascii=False)}\n\n"
def json_error_response(error: AppError) -> Response:
    """将 AppError 转为 Flask JSON 响应"""
    return jsonify({
        "error": {
            "message": error.message,
            "type": type(error).__name__,
            "details": error.details,
        }
    }), error.status_code
def register_error_handlers(app):
    """在 Flask app 上注册统一异常处理器"""
    @app.errorhandler(AppError)
    def handle_app_error(error):
        return json_error_response(error)
    @app.errorhandler(400)
    def handle_bad_request(e):
        return jsonify({
            "error": {"message": "请求格式不正确", "type": "BadRequest"}
        }), 400
    @app.errorhandler(404)
    def handle_not_found(e):
        return jsonify({
            "error": {"message": "请求的资源不存在", "type": "NotFound"}
        }), 404
    @app.errorhandler(405)
    def handle_method_not_allowed(e):
        return jsonify({
            "error": {"message": "请求方法不允许", "type": "MethodNotAllowed"}
        }), 405
    @app.errorhandler(500)
    def handle_internal_error(e):
        return jsonify({
            "error": {"message": "服务器内部错误", "type": "InternalError"}
        }), 500
    return app
-- 文件结束：errors.py --
  文件：validators.py  (84 行, 2.6 KB)
    1  # -*- coding: utf-8 -*-
    2  """
```

```python
请求校验模块
"""
import re
from typing import Optional
from errors import ValidationError
def validate_question(question: Optional[str]) -> str:
    """校验用户提问"""
    if not question:
        raise ValidationError("问题不能为空", field="question")
    q = question.strip()
    if len(q) > 2000:
        raise ValidationError("问题长度不能超过 2000 字符", field="question")
    if len(q) < 1:
        raise ValidationError("问题不能为空", field="question")
    return q
def validate_item(item: Optional[dict]) -> dict:
    """校验意象解析请求中的 item"""
    if not item or not isinstance(item, dict):
        raise ValidationError("缺少诗歌意象数据", field="item")
    i = item.get("文本") or item.get("txt") or ""
    line = item.get("line") or ""
    if not i and not line:
        raise ValidationError("缺少诗句或意象上下文", field="item")
    return item
def validate_history(history: Optional[list]) -> list:
    """校验对话历史"""
    if not history or not isinstance(history, list):
        return []
    valid = []
    for h in history:
        if isinstance(h, dict) and h.get("role") in ("user", "assistant") and h.get("content"):
            valid.append({
                "role": h["role"],
                "content": str(h["content"])[:5000]
            })
    return valid
def validate_poem_id(poem_id: Optional[str]) -> str:
    """校验诗歌编号"""
    if not poem_id or not poem_id.strip():
        raise ValidationError("诗歌编号不能为空", field="poem_id")
    pid = poem_id.strip()
    if not re.match(r"^P?\d{2,6}$", pid):
        raise ValidationError(f"诗歌编号格式不正确: {pid}", field="poem_id")
    return pid
def validate_filename(filename: Optional[str], allowed_exts=None) -> str:
    """校验导出文件名"""
    if not filename or not filename.strip():
        return "export"
    allowed = allowed_exts or [".csv", ".json", ".xlsx"]
    name = filename.strip()
    has_ext = any(name.lower().endswith(ext) for ext in allowed)
    if not has_ext:
        raise ValidationError(
            f"文件名必须包含合法后缀 ({', '.join(allowed)})",
            field="filename"
        )
    if re.search(r"[\\/:*?\"<>|]", name):
        raise ValidationError("文件名包含非法字符", field="filename")
    return name
def sanitize_html(text: str) -> str:
    """简单的 HTML 转义"""
    if not text:
        return ""
    return (text.replace("&", "&amp;")
```

```
                .replace("<", "&lt;")
                .replace(">", "&gt;")
                .replace('"', "&quot;")
                .replace("'", "&#39;"))
```

-- 文件结束：validators.py --

文件：utils.py （146 行, 3.8 KB）

```
# -*- coding: utf-8 -*-
"""
通用工具函数模块 — 文本处理、格式化、系统辅助
"""
import json
import re
import math
import hashlib
from typing import List, Optional, Any
from datetime import datetime
def truncate(text: str, max_len: int = 100, suffix: str = "...") -> str:
    """截断字符串到指定长度"""
    if not text:
        return ""
    if len(text) <= max_len:
        return text
    return text[:max_len].rstrip() + suffix
def extract_numbers(text: str) -> List[int]:
    """从文本中提取所有整数"""
    return [int(x) for x in re.findall(r"\d+", text)]
def is_chinese_char(ch: str) -> bool:
    """判断是否为中文字符"""
    if len(ch) != 1:
        return False
    cp = ord(ch)
    return (
        0x4E00 <= cp <= 0x9FFF
        or 0x3400 <= cp <= 0x4DBF
        or 0x20000 <= cp <= 0x2A6DF
        or 0x2A700 <= cp <= 0x2B73F
    )
def chinese_ratio(text: str) -> float:
    """计算字符串中中文字符占比"""
    if not text:
        return 0.0
    chinese_chars = sum(1 for ch in text if is_chinese_char(ch))
    return round(chinese_chars / len(text), 4)
def split_sentences(text: str) -> List[str]:
    """按中文句号、问号、感叹号、分句"""
    if not text:
        return []
    parts = re.split(r"([。！？\n])", text)
    sentences = []
    buf = ""
    for part in parts:
        buf += part
        if part in ("。", "！", "？", "\n"):
            buf = buf.strip()
            if buf:
                sentences.append(buf)
                buf = ""
    buf = buf.strip()
    if buf:
        sentences.append(buf)
    return sentences
def format_file_size(size_bytes: int) -> str:
    """格式化文件大小（B/KB/MB/GB）"""
    if size_bytes <= 0:
```

```
        return "0 B"
    units = ["B", "KB", "MB", "GB"]
    i = int(math.floor(math.log(size_bytes, 1024))) if size_bytes > 0 else 0
    i = min(i, len(units) - 1)
    size = size_bytes / (1024 ** i)
    return f"{size:.1f} {units[i]}"
def format_timestamp(ts: float, fmt: str = "%Y-%m-%d %H:%M:%S") -> str:
    """格式化时间戳"""
    return datetime.fromtimestamp(ts).strftime(fmt)
def safe_get(data: dict, key: str, default: Any = "") -> Any:
    """安全获取字典值"""
    if not isinstance(data, dict):
        return default
    value = data.get(key)
    if value is None:
        return default
    return value
def dict_pick(data: dict, keys: List[str]) -> dict:
    """从字典中提取指定键的子集"""
    return {k: data[k] for k in keys if k in data}
def dict_omit(data: dict, keys: List[str]) -> dict:
    """从字典中排除指定键"""
    return {k: v for k, v in data.items() if k not in keys}
def md5_hash(text: str) -> str:
    """计算 MD5 哈希"""
    return hashlib.md5(text.encode("utf-8")).hexdigest()
def safe_json_loads(text: str, default: Any = None) -> Any:
    """安全解析 JSON，失败返回默认值"""
    try:
        return json.loads(text)
    except (json.JSONDecodeError, TypeError):
        return default
def list_chunk(items: list, size: int) -> List[list]:
    """将列表按指定大小分块"""
    return [items[i : i + size] for i in range(0, len(items), size)]
def deduplicate_by_key(items: List[dict], key: str) -> List[dict]:
    """按字典的某个键去重（保留第一个）"""
    seen = set()
    result = []
    for item in items:
        val = item.get(key)
        if val not in seen:
            seen.add(val)
            result.append(item)
    return result
def merge_dicts(base: dict, override: dict) -> dict:
    """合并字典，override 覆盖 base"""
    result = base.copy()
    result.update(override)
    return result
def normalize_whitespace(text: str) -> str:
    """规范化空白字符"""
    if not text:
        return ""
    return re.sub(r"\s+", " ", text).strip()
-- 文件结束：utils.py --
文件：middleware.py  (188 行, 5.6 KB)
# -*- coding: utf-8 -*-
"""
中间件模块 — 请求标识、计时、CORS、日志追踪
"""
import time
import uuid
import threading
```

```
from functools import wraps
from flask import request, g, jsonify
from config import BASE_DIR
from logger import get_logger
logger = get_logger("middleware")
# ─── 请求标识 ───
class RequestContext:
    """线程级请求上下文"""
    def __init__(self):
        self.request_id = None
        self.start_time = None
        self.client_ip = None
        self.endpoint = None
        self.method = None
    def init_from_flask(self):
        self.request_id = request.headers.get("X-Request-ID") or \
                          request.args.get("_rid") or \
                          uuid.uuid4().hex[:12]
        self.start_time = time.time()
        self.client_ip = request.remote_addr or "unknown"
        self.endpoint = request.endpoint or request.path
        self.method = request.method
        return self
    def elapsed(self):
        if self.start_time:
            return round((time.time() - self.start_time) * 1000, 2)
        return 0
    def to_dict(self):
        return {
            "request_id": self.request_id,
            "client_ip": self.client_ip,
            "endpoint": self.endpoint,
            "method": self.method,
            "elapsed_ms": self.elapsed(),
        }
_request_ctx = threading.local()
def get_request_ctx() -> RequestContext:
    if not hasattr(_request_ctx, "current"):
        _request_ctx.current = RequestContext()
    return _request_ctx.current
# ─── 装饰器 ───
def request_context_middleware(app):
    """为 Flask app 注册请求生命周期钩子"""
    @app.before_request
    def before_request():
        ctx = get_request_ctx()
        ctx.init_from_flask()
        g.request_id = ctx.request_id
        logger.info(f"[{ctx.request_id}] {ctx.method} {request.path} from {ctx.client_ip}")
    @app.after_request
    def after_request(response):
        ctx = get_request_ctx()
        elapsed = ctx.elapsed()
        status = response.status_code
        logger.info(f"[{ctx.request_id}] {status} ({elapsed}ms)")
        response.headers.set("X-Request-ID", ctx.request_id)
        response.headers.set("X-Response-Time", f"{elapsed}ms")
        return response
    return app
def cors_middleware(app, origins=None):
    """为 Flask app 添加 CORS 头"""
    if origins is None:
        origins = ["*"]
    @app.after_request
```

```
    def add_cors_headers(response):
        origin = request.headers.get("Origin", "")
        if "*" in origins or origin in origins:
            response.headers.set("Access-Control-Allow-Origin", origin)
            response.headers.set("Access-Control-Allow-Methods",
                                 "GET, POST, PUT, DELETE, OPTIONS")
            response.headers.set("Access-Control-Allow-Headers",
                                 "Content-Type, Authorization, X-Request-ID")
            response.headers.set("Access-Control-Allow-Credentials", "true")
        return response
    return app
def rate_limit(max_per_minute=60):
    """简易速率限制装饰器（进程内计数）"""
    records = {}
    def decorator(f):
        @wraps(f)
        def wrapper(*args, **kwargs):
            now = time.time()
            ip = request.remote_addr or "unknown"
            window_start = now - 60
            if ip not in records:
                records[ip] = []
            records[ip] = [t for t in records[ip] if t > window_start]
            if len(records[ip]) >= max_per_minute:
                logger.warning(f"速率限制触发: {ip}")
                return jsonify({"error": "请求过于频繁，请稍后再试"}), 429
            records[ip].append(now)
            return f(*args, **kwargs)
        return wrapper
    return decorator
def timer(f):
    """函数执行计时装饰器"""
    @wraps(f)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = f(*args, **kwargs)
        elapsed = round((time.time() - start) * 1000, 2)
        logger.debug(f"{f.__name__} 执行耗时: {elapsed}ms")
        return result
    return wrapper
class PerformanceTracker:
    """性能追踪器——记录接口调用统计"""
    def __init__(self):
        self._lock = threading.Lock()
        self._stats = {}
    def record(self, endpoint: str, elapsed_ms: float, status: int):
        with self._lock:
            if endpoint not in self._stats:
                self._stats[endpoint] = {
                    "count": 0, "total_time": 0, "max_time": 0,
                    "min_time": float("inf"), "error_count": 0,
                }
            s = self._stats[endpoint]
            s["count"] += 1
            s["total_time"] += elapsed_ms
            s["max_time"] = max(s["max_time"], elapsed_ms)
            s["min_time"] = min(s["min_time"], elapsed_ms)
            if status >= 400:
                s["error_count"] += 1
    def summary(self):
        result = {}
        with self._lock:
            for ep, s in self._stats.items():
                avg = round(s["total_time"] / s["count"], 2) if s["count"] else 0
```

```
                err_rate = round(s["error_count"] / s["count"] * 100, 1) if s["count"] else 0
                result[ep] = {
                    "calls": s["count"],
                    "avg_ms": avg,
                    "max_ms": round(s["max_time"], 2),
                    "min_ms": round(s["min_time"], 2) if s["min_time"] != float("inf") else 0,
                    "errors": s["error_count"],
                    "error_rate_pct": err_rate,
                }
        return result
    def reset(self):
        with self._lock:
            self._stats.clear()
tracker = PerformanceTracker()
```

-- 文件结束：middleware.py --

文件：cache.py (144 行, 4.3 KB)

```
# -*- coding: utf-8 -*-
"""
缓存模块 — 文件缓存 + 内存缓存，支持 TTL
"""
import json
import os
import time
import threading
from config import CACHE_DIR, CACHE_DEFAULT_TTL, CACHE_ENABLED
from logger import get_logger
logger = get_logger("cache")
class MemoryCache:
    """线程安全的内存缓存"""
    def __init__(self, default_ttl: int = CACHE_DEFAULT_TTL):
        self._store = {}
        self._lock = threading.RLock()
        self.default_ttl = default_ttl
    def get(self, key: str):
        if not CACHE_ENABLED:
            return None
        with self._lock:
            entry = self._store.get(key)
            if entry is None:
                return None
            if entry["expires"] is not None and time.time() > entry["expires"]:
                del self._store[key]
                return None
            return entry["value"]
    def set(self, key: str, value, ttl: int = None):
        if not CACHE_ENABLED:
            return
        expires = None
        if ttl is None and self.default_ttl > 0:
            expires = time.time() + self.default_ttl
        elif ttl and ttl > 0:
            expires = time.time() + ttl
        with self._lock:
            self._store[key] = {"value": value, "expires": expires}
    def delete(self, key: str):
        with self._lock:
            self._store.pop(key, None)
    def clear(self):
        with self._lock:
            self._store.clear()
    def size(self) -> int:
        with self._lock:
            return len(self._store)
    def keys(self):
```

```
        with self._lock:
            return list(self._store.keys())
class FileCache:
    """文件缓存 — 持久化到磁盘"""
    def __init__(self, cache_dir: str = None, default_ttl: int = CACHE_DEFAULT_TTL):
        self.cache_dir = cache_dir or CACHE_DIR
        self.default_ttl = default_ttl
        self._lock = threading.RLock()
        if not os.path.exists(self.cache_dir):
            os.makedirs(self.cache_dir, exist_ok=True)
    def _path(self, key: str) -> str:
        safe = key.replace("/", "_").replace("\\", "_").replace(":", "_")
        return os.path.join(self.cache_dir, f"{safe}.json")
    def get(self, key: str):
        if not CACHE_ENABLED:
            return None
        path = self._path(key)
        try:
            with open(path, "r", encoding="utf-8") as f:
                entry = json.load(f)
            if entry.get("expires") and time.time() > entry["expires"]:
                os.remove(path)
                return None
            return entry["value"]
        except (FileNotFoundError, json.JSONDecodeError, KeyError):
            return None
    def set(self, key: str, value, ttl: int = None):
        if not CACHE_ENABLED:
            return
        expires = None
        if ttl is None and self.default_ttl > 0:
            expires = time.time() + self.default_ttl
        elif ttl and ttl > 0:
            expires = time.time() + ttl
        with self._lock:
            path = self._path(key)
            with open(path, "w", encoding="utf-8") as f:
                json.dump({"value": value, "expires": expires}, f, ensure_ascii=False)
    def delete(self, key: str):
        path = self._path(key)
        try:
            os.remove(path)
        except FileNotFoundError:
            pass
    def clear(self):
        with self._lock:
            for fname in os.listdir(self.cache_dir):
                if fname.endswith(".json"):
                    os.remove(os.path.join(self.cache_dir, fname))
    def size(self) -> int:
        return len([f for f in os.listdir(self.cache_dir) if f.endswith(".json")])
# 全局单例
memory_cache = MemoryCache()
file_cache = FileCache()
class cached:
    """装饰器：缓存函数返回值"""
    def __init__(self, ttl: int = None, cache_backend: str = "memory"):
        self.ttl = ttl
        self.cache = memory_cache if cache_backend == "memory" else file_cache
    def __call__(self, func):
        def wrapper(*args, **kwargs):
            key = f"{func.__module__}:{func.__name__}:{repr(args)}:{repr(sorted(kwargs.items()))}"
            result = self.cache.get(key)
            if result is not None:
```

```
                return result
            result = func(*args, **kwargs)
            self.cache.set(key, result, ttl=self.ttl)
            return result
        return wrapper
-- 文件结束：cache.py --
文件：models.py  (193 行, 5.1 KB)
# -*- coding: utf-8 -*-
"""
数据模型层 — 诗歌、意象、分析单元的数据类定义
"""
import json
from typing import List, Optional, Dict, Any
from dataclasses import dataclass, field, asdict
@dataclass
class PoemLine:
    """诗行"""
    诗行编号: str = ""
    原文: str = ""
    注释: str = ""
    def to_dict(self):
        return asdict(self)
@dataclass
class AnalysisUnit:
    """分析单元（意象标注单元）"""
    单元编号: str = ""
    诗行编号: str = ""
    文本: str = ""
    行内位置: str = ""
    词性: str = ""
    成分类型: str = ""
    是否意象: str = "0"
    大类编码: str = ""
    子类编码: str = ""
    感知通道: str = ""
    素材类型: str = ""
    内部结构: str = ""
    指涉来源: str = ""
    表现功能: str = ""
    文化流通性: str = ""
    跨文化性: str = ""
    认知强度: str = ""
    核心意象: str = ""
    结构功能组: str = ""
    情感极性: str = ""
    情感类别: str = ""
    情感置信度: str = ""
    def to_dict(self):
        return asdict(self)
    @property
    def is_image(self) -> bool:
        return self.是否意象 == "1"
@dataclass
class EmotionTrajectory:
    """情感轨迹"""
    诗行编号: str = ""
    诗行原文: str = ""
    平均情感极性: str = ""
    主导情感: str = ""
    情感波动值: str = ""
    def to_dict(self):
        return asdict(self)
@dataclass
class ImageRelation:
```

```
    """意象关系"""
    关系编号: str = ""
    来源单元编号: str = ""
    来源文本: str = ""
    目标单元编号: str = ""
    目标文本: str = ""
    关系类型: str = ""
    关系强度: str = ""
    def to_dict(self):
        return asdict(self)
@dataclass
class Poem:
    """诗歌完整数据模型"""
    诗歌编号: str = ""
    标题: str = ""
    作者: str = ""
    朝代: str = "唐"
    分类标签: str = ""
    体裁: str = ""
    原文: str = ""
    诗行: List[PoemLine] = field(default_factory=list)
    分析单元: List[AnalysisUnit] = field(default_factory=list)
    情感轨迹: List[EmotionTrajectory] = field(default_factory=list)
    意象关系: List[ImageRelation] = field(default_factory=list)
    def to_dict(self):
        return {
            "诗歌编号": self.诗歌编号,
            "标题": self.标题,
            "作者": self.作者,
            "分类标签": self.分类标签,
            "体裁": self.体裁,
            "原文": self.原文,
            "诗行": [l.to_dict() for l in self.诗行],
            "分析单元": [u.to_dict() for u in self.分析单元],
            "情感轨迹": [t.to_dict() for t in self.情感轨迹],
            "意象关系": [r.to_dict() for r in self.意象关系],
        }
    @property
    def image_units(self) -> List[AnalysisUnit]:
        return [u for u in self.分析单元 if u.is_image]
    @property
    def image_count(self) -> int:
        return len(self.image_units)
    @property
    def summary(self) -> str:
        return f"《{self.标题}》{self.作者} ({self.诗歌编号}) — {self.image_count}个意象"
    @classmethod
    def from_dict(cls, data: dict) -> "Poem":
        lines = [PoemLine(**l) for l in data.get("诗行", []) if isinstance(l, dict)]
        units = [AnalysisUnit(**u) for u in data.get("分析单元", []) if isinstance(u, dict)]
        trajs = [EmotionTrajectory(**t) for t in data.get("情感轨迹", []) if isinstance(t, dict)]
        rels = [ImageRelation(**r) for r in data.get("意象关系", []) if isinstance(r, dict)]
        return cls(
            诗歌编号=data.get("诗歌编号", ""),
            标题=data.get("标题", ""),
            作者=data.get("作者", ""),
            分类标签=data.get("分类标签", ""),
            体裁=data.get("体裁", ""),
            原文=data.get("原文", ""),
            诗行=lines, 分析单元=units,
            情感轨迹=trajs, 意象关系=rels,
        )
@dataclass
class TracebackItem:
```

```
    """溯源表条目（前台展示用）"""
    poem_id: str = ""
    title: str = ""
    author: str = ""
    genre: str = ""
    cat: str = ""
    txt: str = ""
    text: str = ""
    dimensions: str = ""
    emotion: str = ""
    emo_cat: str = ""
    line: str = ""
    id: str = ""
    词性: str = ""
    成分类型: str = ""
    感知通道: str = ""
    素材类型: str = ""
    内部结构: str = ""
    指涉来源: str = ""
    表现功能: str = ""
    文化流通性: str = ""
    跨文化性: str = ""
    认知强度: str = ""
    核心意象: str = ""
    结构功能组: str = ""
    情感极性: str = ""
    情感类别: str = ""
    情感置信度: str = ""
    大类编码: str = ""
    子类编码: str = ""
    def to_dict(self):
        return asdict(self)
@dataclass
class AnalysisResult:
    """解析结果"""
    item: Dict[str, Any] = field(default_factory=dict)
    messages: List[Dict] = field(default_factory=list)
    done: bool = False
    is_streaming: bool = False
    def to_dict(self):
        return {
            "done": self.done,
            "is_streaming": self.is_streaming,
            "message_count": len(self.messages),
        }
-- 文件结束：models.py --
文件：analytics.py  (313 行, 11.0 KB)
# -*- coding: utf-8 -*-
"""
数据统计分析服务 — 多维度聚合统计
"""
import json
from collections import Counter, defaultdict
from typing import List, Dict, Any
from config import CATEGORY_NAME_MAP, CATEGORY_MAJOR_MAP
from logger import get_logger
logger = get_logger("analytics")
class StatsService:
    """意象数据统计分析服务"""
    def __init__(self, traceback_data: List[Dict] = None):
        self.data = traceback_data or []
        self._cache = {}
    def set_data(self, data: List[Dict]):
        self.data = data
```

```
        self._cache.clear()
    # ——— 基础统计 ———
    def total_images(self) -> int:
        return len(self.data)
    def total_poems(self) -> int:
        return len({item.get("poem_id") for item in self.data if item.get("poem_id")})
    def total_authors(self) -> int:
        return len({item.get("author") for item in self.data if item.get("author")})
    # ——— 意象频次统计 ———
    def top_images(self, n: int = 50) -> List[Dict]:
        counter = Counter()
        for item in self.data:
            txt = item.get("文本") or item.get("txt") or ""
            if txt:
                counter[txt] += 1
        return [{"text": txt, "count": cnt} for txt, cnt in counter.most_common(n)]
    # ——— 分类统计 ———
    def category_distribution(self) -> List[Dict]:
        counter = Counter()
        for item in self.data:
            cat = item.get("cat", "未分类")
            counter[cat] += 1
        total = sum(counter.values()) or 1
        return [
            {"category": cat, "count": cnt, "percentage": round(cnt / total * 100, 2)}
            for cat, cnt in counter.most_common()
        ]
    def major_category_distribution(self) -> List[Dict]:
        counter = Counter()
        for item in self.data:
            code = str(item.get("大类编码", ""))
            major = CATEGORY_MAJOR_MAP.get(code, "其他")
            counter[major] += 1
        total = sum(counter.values()) or 1
        return [
            {"category": cat, "count": cnt, "percentage": round(cnt / total * 100, 2)}
            for cat, cnt in counter.most_common()
        ]
    def sub_category_distribution(self, major_code: str = None) -> List[Dict]:
        counter = Counter()
        for item in self.data:
            code = str(item.get("子类编码", ""))
            if major_code and not code.startswith(major_code):
                continue
            cat = CATEGORY_NAME_MAP.get(code, f"未知子类 ({code})")
            counter[cat] += 1
        total = sum(counter.values()) or 1
        return [
            {"category": cat, "count": cnt, "percentage": round(cnt / total * 100, 2)}
            for cat, cnt in counter.most_common()
        ]
    # ——— 诗人统计 ———
    def author_statistics(self) -> List[Dict]:
        author_images = defaultdict(list)
        for item in self.data:
            author = item.get("author") or item.get("作者") or "未知"
            txt = item.get("文本") or item.get("txt") or ""
            if txt:
                author_images[author].append(txt)
        results = []
        for author, images in sorted(author_images.items(), key=lambda x: -len(x[1])):
            unique_images = set(images)
            top = Counter(images).most_common(5)
            results.append({
```

```
                "author": author,
                "image_count": len(images),
                "unique_image_count": len(unique_images),
                "poem_count": len({
                    item.get("poem_id") for item in self.data
                    if (item.get("author") or item.get("作者")) == author
                }),
                "top_images": [{"text": t, "count": c} for t, c in top],
            })
        return results
    # ——— 情感分析 ———
    def emotion_distribution(self) -> List[Dict]:
        counter = Counter()
        for item in self.data:
            emo = item.get("情感类别") or item.get("emo_cat") or "未知"
            counter[emo] += 1
        total = sum(counter.values()) or 1
        return [
            {"emotion": emo, "count": cnt, "percentage": round(cnt / total * 100, 2)}
            for emo, cnt in counter.most_common()
        ]
    def emotion_polarity_distribution(self) -> List[Dict]:
        counter = Counter()
        for item in self.data:
            pol = item.get("情感极性") or "未标注"
            counter[pol] += 1
        total = sum(counter.values()) or 1
        return [
            {"polarity": pol, "count": cnt, "percentage": round(cnt / total * 100, 2)}
            for pol, cnt in counter.most_common()
        ]
    # ——— 感知通道统计 ———
    def perception_channel_distribution(self) -> List[Dict]:
        counter = Counter()
        for item in self.data:
            ch = item.get("感知通道") or "未标注"
            counter[ch] += 1
        total = sum(counter.values()) or 1
        return [
            {"channel": ch, "count": cnt, "percentage": round(cnt / total * 100, 2)}
            for ch, cnt in counter.most_common()
        ]
    # ——— 诗词体裁统计 ———
    def genre_distribution(self) -> List[Dict]:
        """按诗歌体裁分类统计"""
        counter = Counter()
        for item in self.data:
            genre = item.get("genre", "未分类")
            if genre:
                counter[genre] += 1
        total = sum(counter.values()) or 1
        return [
            {"genre": g, "count": cnt, "percentage": round(cnt / total * 100, 2)}
            for g, cnt in counter.most_common()
        ]
    # ——— 内部结构统计 ———
    def internal_structure_distribution(self) -> List[Dict]:
        """内部结构维度统计"""
        counter = Counter()
        for item in self.data:
            val = item.get("内部结构") or "未标注"
            counter[val] += 1
        total = sum(counter.values()) or 1
        return [
```

```
            {"structure": s, "count": cnt, "percentage": round(cnt / total * 100, 2)}
            for s, cnt in counter.most_common()
        ]
    # ——— 表现功能统计 ———
    def function_distribution(self) -> List[Dict]:
        """表现功能维度统计"""
        counter = Counter()
        for item in self.data:
            val = item.get("表现功能") or "未标注"
            counter[val] += 1
        total = sum(counter.values()) or 1
        return [
            {"function": f, "count": cnt, "percentage": round(cnt / total * 100, 2)}
            for f, cnt in counter.most_common()
        ]
    # ——— 核心意象统计 ———
    def core_image_distribution(self) -> List[Dict]:
        """核心意象标注统计"""
        counter = Counter()
        for item in self.data:
            val = item.get("核心意象") or "未标注"
            counter[val] += 1
        total = sum(counter.values()) or 1
        return [
            {"core_image": c, "count": cnt, "percentage": round(cnt / total * 100, 2)}
            for c, cnt in counter.most_common()
        ]
    # ——— 指涉来源统计 ———
    def reference_source_distribution(self) -> List[Dict]:
        """指涉来源维度统计"""
        counter = Counter()
        for item in self.data:
            val = item.get("指涉来源") or "未标注"
            counter[val] += 1
        total = sum(counter.values()) or 1
        return [
            {"source": s, "count": cnt, "percentage": round(cnt / total * 100, 2)}
            for s, cnt in counter.most_common()
        ]
    # ——— 认知强度统计 ———
    def cognitive_intensity_distribution(self) -> List[Dict]:
        """认知强度维度分布"""
        counter = Counter()
        for item in self.data:
            val = item.get("认知强度") or "未标注"
            counter[val] += 1
        total = sum(counter.values()) or 1
        return [
            {"intensity": c, "count": cnt, "percentage": round(cnt / total * 100, 2)}
            for c, cnt in counter.most_common()
        ]
    # ——— 素材类型统计 ———
    def material_type_distribution(self) -> List[Dict]:
        """素材类型维度统计"""
        counter = Counter()
        for item in self.data:
            val = item.get("素材类型") or "未标注"
            counter[val] += 1
        total = sum(counter.values()) or 1
        return [
            {"material": m, "count": cnt, "percentage": round(cnt / total * 100, 2)}
            for m, cnt in counter.most_common()
        ]
    # ——— 跨文化性统计 ———
```

```
    def cross_cultural_distribution(self) -> List[Dict]:
        """跨文化性维度统计"""
        counter = Counter()
        for item in self.data:
            val = item.get("跨文化性") or "未标注"
            counter[val] += 1
        total = sum(counter.values()) or 1
        return [
            {"level": c, "count": cnt, "percentage": round(cnt / total * 100, 2)}
            for c, cnt in counter.most_common()
        ]
    # ——— 综合报告 ———
    def summary_report(self) -> Dict:
        return {
            "total_images": self.total_images(),
            "total_poems": self.total_poems(),
            "total_authors": self.total_authors(),
            "category_count": len(self.category_distribution()),
            "emotion_categories": len(self.emotion_distribution()),
            "top_images": self.top_images(20),
            "category_stats": self.category_distribution(),
            "major_category_stats": self.major_category_distribution(),
            "emotion_stats": self.emotion_distribution(),
            "emotion_polarity_stats": self.emotion_polarity_distribution(),
            "perception_channel_stats": self.perception_channel_distribution(),
            "genre_stats": self.genre_distribution(),
            "internal_structure_stats": self.internal_structure_distribution(),
            "function_stats": self.function_distribution(),
            "core_image_stats": self.core_image_distribution(),
            "reference_source_stats": self.reference_source_distribution(),
            "cognitive_intensity_stats": self.cognitive_intensity_distribution(),
            "material_type_stats": self.material_type_distribution(),
            "cross_cultural_stats": self.cross_cultural_distribution(),
        }
    # ——— 跨表分析 ———
    def cross_analysis(self) -> Dict:
        """情感 × 分类域 交叉分析"""
        cross = defaultdict(lambda: defaultdict(int))
        for item in self.data:
            emo = item.get("情感类别") or "未知"
            cat = item.get("cat", "未分类")
            cross[cat][emo] += 1
        result = {}
        for cat, emotions in cross.items():
            total = sum(emotions.values())
            result[cat] = [
                {"emotion": emo, "count": cnt, "percentage": round(cnt / total * 100, 2)}
                for emo, cnt in sorted(emotions.items(), key=lambda x: -x[1])
            ]
        return result
def build_analytics_api(stats_service: StatsService) -> Dict:
    """生成供 API 返回的分析数据"""
    return {
        "summary": stats_service.summary_report(),
        "cross_analysis": stats_service.cross_analysis(),
    }
-- 文件结束：analytics.py --
文件：export_service.py  (118 行, 4.0 KB)
# -*- coding: utf-8 -*-
"""
数据导出服务 — CSV / JSON / 报告导出
"""
import csv
import json
```

```python
import os
import time
from typing import List, Dict, Optional
from config import EXPORT_DIR, EXPORT_CSV_ENCODING
from errors import ExportError
from logger import get_logger
logger = get_logger("export")
def ensure_export_dir():
    if not os.path.exists(EXPORT_DIR):
        os.makedirs(EXPORT_DIR, exist_ok=True)
class ExportService:
    """数据导出服务"""
    @staticmethod
    def export_traceback_to_csv(data: List[Dict], filename: str = None) -> str:
        """将溯源数据导出为 CSV"""
        ensure_export_dir()
        filename = filename or f"traceback_export_{int(time.time())}.csv"
        filepath = os.path.join(EXPORT_DIR, filename)
        if not data:
            raise ExportError("没有数据可导出")
        fieldnames = [
            "poem_id", "title", "genre", "cat", "txt",
            "emotion", "line", "词性", "成分类型", "感知通道",
            "素材类型", "内部结构", "指涉来源", "表现功能",
            "文化流通性", "跨文化性", "认知强度", "核心意象",
            "结构功能组", "情感极性", "情感类别", "情感置信度",
            "大类编码", "子类编码",
        ]
        try:
            with open(filepath, "w", encoding=EXPORT_CSV_ENCODING, newline="") as f:
                writer = csv.DictWriter(f, fieldnames=fieldnames, extrasaction="ignore")
                writer.writeheader()
                for row in data:
                    writer.writerow(row)
            logger.info(f"CSV 导出成功: {filepath} ({len(data)} 条)")
            return filepath
        except IOError as e:
            raise ExportError(f"CSV 写入失败: {e}")
    @staticmethod
    def export_to_json(data, filename: str = None, indent: int = 2) -> str:
        """将数据导出为 JSON"""
        ensure_export_dir()
        filename = filename or f"export_{int(time.time())}.json"
        filepath = os.path.join(EXPORT_DIR, filename)
        try:
            with open(filepath, "w", encoding="utf-8") as f:
                json.dump(data, f, ensure_ascii=False, indent=indent)
            logger.info(f"JSON 导出成功: {filepath}")
            return filepath
        except (IOError, TypeError) as e:
            raise ExportError(f"JSON 写入失败: {e}")
    @staticmethod
    def export_summary_report(report: Dict, filename: str = None) -> str:
        """导出统计分析报告（JSON 格式）"""
        return ExportService.export_to_json(report, filename or f"report_{int(time.time())}.json")
    @staticmethod
    def export_filtered_slice(
        data: List[Dict],
        start: int = 0,
        end: int = None,
        filename: str = None,
    ) -> str:
        """导出数据子集"""
        subset = data[start:end] if end else data[start:]
```

```
        return ExportService.export_to_json(
            {"total": len(data), "start": start, "end": end or len(data),
             "count": len(subset), "data": subset},
            filename or f"slice_{start}_{end or len(data)}.json",
        )
    @staticmethod
    def list_exports() -> List[Dict]:
        """列出已导出的文件"""
        ensure_export_dir()
        files = []
        for fname in os.listdir(EXPORT_DIR):
            fpath = os.path.join(EXPORT_DIR, fname)
            if os.path.isfile(fpath):
                files.append({
                    "name": fname,
                    "path": fpath,
                    "size": os.path.getsize(fpath),
                    "mtime": os.path.getmtime(fpath),
                })
        return sorted(files, key=lambda x: -x["mtime"])
    @staticmethod
    def clear_exports() -> int:
        """清空导出目录"""
        ensure_export_dir()
        count = 0
        for fname in os.listdir(EXPORT_DIR):
            fpath = os.path.join(EXPORT_DIR, fname)
            if os.path.isfile(fpath):
                os.remove(fpath)
                count += 1
        logger.info(f"清空导出目录: 移除 {count} 个文件")
        return count
-- 文件结束：export_service.py --
文件：preprocessor.py  (149 行, 5.1 KB)
# -*- coding: utf-8 -*-
"""
数据预处理模块 — JSON 清洗、格式校验、批量转换
"""
import json
import os
import re
import glob
import shutil
from typing import List, Dict, Optional, Tuple
from config import POEM_JSON_DIR, BASE_DIR
from logger import get_logger
from models import Poem, AnalysisUnit
logger = get_logger("preprocessor")
def clean_json_content(content: str) -> str:
    """清理 JSON 文本中的 Markdown 包裹和不规范字符"""
    content = re.sub(r"```json\s*", "", content, flags=re.MULTILINE)
    content = re.sub(r"```\s*", "", content, flags=re.MULTILINE)
    content = re.sub(r",\s*}", "}", content)
    content = re.sub(r",\s*]", "]", content)
    content = content.strip()
    return content
def validate_poem_json(data: dict) -> List[str]:
    """验证诗歌 JSON 结构，返回错误列表"""
    errors = []
    if not isinstance(data, dict):
        return ["根节点不是字典"]
    poems = data.get("诗歌集", [data]) if isinstance(data, dict) else data
    if not isinstance(poems, list):
        poems = [poems]
```

```
    for i, poem in enumerate(poems):
        prefix = f"[诗歌 #{i+1}]"
        if not poem.get("标题"):
            errors.append(f"{prefix} 缺少标题字段")
        if not poem.get("诗歌编号") and not poem.get("编号"):
            errors.append(f"{prefix} 缺少诗歌编号")
        if not poem.get("原文") and not poem.get("诗行"):
            errors.append(f"{prefix} 缺少原文或诗行")
        if "分析单元" in poem:
            for j, unit in enumerate(poem["分析单元"]):
                if not isinstance(unit, dict):
                    errors.append(f"{prefix} 分析单元 #{j+1} 不是字典")
                    continue
                if not unit.get("文本"):
                    errors.append(f"{prefix} 分析单元 #{j+1} 缺少文本")
                if not unit.get("诗行编号"):
                    errors.append(f"{prefix} 分析单元 #{j+1} 缺少诗行编号")
    return errors
def parse_json_file(filepath: str) -> Tuple[Optional[dict], List[str]]:
    """安全解析 JSON 文件，返回 (data, errors)"""
    try:
        with open(filepath, "r", encoding="utf-8") as f:
            content = f.read()
    except IOError as e:
        return None, [f"读取失败: {e}"]
    content = clean_json_content(content)
    try:
        data = json.loads(content)
    except json.JSONDecodeError as e:
        return None, [f"JSON 解析失败: {e}"]
    errors = validate_poem_json(data)
    return data, errors
def batch_validate(directory: str = None) -> Dict:
    """批量校验目录下所有 JSON 文件"""
    directory = directory or POEM_JSON_DIR
    if not os.path.exists(directory):
        return {"total": 0, "valid": 0, "invalid": 0, "details": []}
    files = glob.glob(os.path.join(directory, "*.json"))
    results = {"total": len(files), "valid": 0, "invalid": 0, "details": []}
    for fpath in sorted(files):
        fname = os.path.basename(fpath)
        data, errors = parse_json_file(fpath)
        if errors:
            results["invalid"] += 1
            results["details"].append({"file": fname, "status": "invalid",
                                       "errors": errors})
            logger.warning(f"校验失败: {fname} — {'; '.join(errors)}")
        else:
            results["valid"] += 1
            results["details"].append({"file": fname, "status": "valid",
                                       "poem_count": len(
                                           data.get("诗歌集", [data])
                                           if isinstance(data, dict) else data
                                       )})
            logger.info(f"校验通过: {fname}")
    return results
def backup_data(directory: str = None) -> str:
    """备份数据目录"""
    src = directory or POEM_JSON_DIR
    if not os.path.exists(src):
        logger.warning(f"备份源目录不存在: {src}")
        return ""
    backup_name = f"backup_poem_json_{__import__('time').time():.0f}"
    dst = os.path.join(BASE_DIR, backup_name)
```

```
    shutil.copytree(src, dst)
    logger.info(f"数据备份完成: {src} -> {dst}")
    return dst
def convert_raw_poem_to_model(poem_dict: dict) -> Poem:
    """将原始 JSON 字典转为 Poem 模型"""
    return Poem.from_dict(poem_dict)
def extract_image_statistics(poems: List[Poem]) -> Dict:
    """从 Poem 模型列表提取意象统计"""
    total_images = 0
    image_texts = []
    author_images = {}
    for poem in poems:
        images = poem.image_units
        total_images += len(images)
        for u in images:
            image_texts.append(u.文本)
        author = poem.作者 or "未知"
        if author not in author_images:
            author_images[author] = []
        author_images[author].extend(u.文本 for u in images)
    from collections import Counter
    top = Counter(image_texts).most_common(20)
    return {
        "total_poems": len(poems),
        "total_images": total_images,
        "unique_images": len(set(image_texts)),
        "top_images": [{"text": t, "count": c} for t, c in top],
        "authors": len(author_images),
    }
```

-- 文件结束：preprocessor.py --

文件：monitor.py （186 行, 5.6 KB）

```
# -*- coding: utf-8 -*-
"""
系统监控模块 — 资源使用、接口统计、健康检查
"""
import os
import time
import threading
from config import BASE_DIR
from logger import get_logger, LoggerMixin
logger = get_logger("monitor")
class SystemMonitor(LoggerMixin):
    """系统资源监控"""
    def __init__(self, interval: int = 60):
        self.interval = interval
        self._running = False
        self._thread = None
        self._snapshots = []
        self._lock = threading.Lock()
    @property
    def disk_usage(self) -> dict:
        """磁盘使用情况"""
        try:
            import shutil
            total, used, free = shutil.disk_usage(BASE_DIR)
            return {
                "total_gb": round(total / (1024**3), 2),
                "used_gb": round(used / (1024**3), 2),
                "free_gb": round(free / (1024**3), 2),
                "usage_pct": round(used / total * 100, 1) if total else 0,
            }
        except Exception:
            return {"error": "无法获取磁盘信息"}
    @property
```

```
    def memory_info(self) -> dict:
        """内存使用信息"""
        try:
            import psutil
            mem = psutil.virtual_memory()
            return {
                "total_gb": round(mem.total / (1024**3), 2),
                "available_gb": round(mem.available / (1024**3), 2),
                "usage_pct": mem.percent,
            }
        except ImportError:
            return {"note": "需安装 psutil (pip install psutil)"}
        except Exception as e:
            return {"error": str(e)}
    @property
    def cpu_info(self) -> dict:
        """CPU 使用信息"""
        try:
            import psutil
            return {
                "cpu_percent": psutil.cpu_percent(interval=0.5),
                "cpu_count": psutil.cpu_count(),
                "load_avg": os.getloadavg() if hasattr(os, "getloadavg") else None,
            }
        except ImportError:
            return {"note": "需安装 psutil"}
        except Exception as e:
            return {"error": str(e)}
    @property
    def python_info(self) -> dict:
        """Python 运行时信息"""
        import sys
        return {
            "version": sys.version,
            "executable": sys.executable,
            "platform": sys.platform,
        }
    @property
    def project_info(self) -> dict:
        """项目文件信息"""
        py_files = 0
        py_lines = 0
        for root, dirs, files in os.walk(BASE_DIR):
            dirs[:] = [d for d in dirs if d not in ("__pycache__", "rag_db",
                                                    "node_modules", ".git")]
            for f in files:
                if f.endswith(".py"):
                    py_files += 1
                    fpath = os.path.join(root, f)
                    try:
                        with open(fpath, "r", encoding="utf-8") as fh:
                            py_lines += sum(1 for _ in fh)
                    except Exception:
                        pass
        return {
            "python_files": py_files,
            "python_lines": py_lines,
            "project_dir": BASE_DIR,
        }
    def snapshot(self) -> dict:
        """获取当前系统快照"""
        snap = {
            "timestamp": time.time(),
            "time_str": time.strftime("%Y-%m-%d %H:%M:%S"),
```

```
            "disk": self.disk_usage,
            "memory": self.memory_info,
            "cpu": self.cpu_info,
            "python": self.python_info,
            "project": self.project_info,
        }
        self.log_debug(f"系统快照已记录")
        return snap
    def collect(self) -> dict:
        """收集并记录快照"""
        snap = self.snapshot()
        with self._lock:
            self._snapshots.append(snap)
            if len(self._snapshots) > 100:
                self._snapshots = self._snapshots[-100:]
        return snap
    def start_background_collection(self):
        """启动后台定时收集"""
        if self._running:
            return
        self._running = True
        self._thread = threading.Thread(target=self._run_loop, daemon=True)
        self._thread.start()
        self.log_info(f"后台监控已启动 (间隔 {self.interval}s)")
    def stop_background_collection(self):
        """停止后台收集"""
        self._running = False
        if self._thread:
            self._thread.join(timeout=5)
            self._thread = None
        self.log_info("后台监控已停止")
    def _run_loop(self):
        while self._running:
            try:
                self.collect()
            except Exception as e:
                self.log_error(f"监控采集异常: {e}")
            time.sleep(self.interval)
    def get_history(self, count: int = 10) -> list:
        """获取最近 N 次快照"""
        with self._lock:
            return self._snapshots[-count:]
    def health_check(self) -> dict:
        """健康检查"""
        snap = self.collect()
        issues = []
        disk = snap.get("disk", {})
        if disk.get("usage_pct", 0) > 90:
            issues.append(f"磁盘使用率过高: {disk['usage_pct']}%")
        mem = snap.get("memory", {})
        if mem.get("usage_pct", 0) > 90:
            issues.append(f"内存使用率过高: {mem['usage_pct']}%")
        return {
            "status": "unhealthy" if issues else "healthy",
            "timestamp": snap["timestamp"],
            "time_str": snap["time_str"],
            "issues": issues,
            "disk_usage_pct": disk.get("usage_pct"),
            "memory_usage_pct": mem.get("usage_pct"),
        }
# 全局单例
system_monitor = SystemMonitor()
def get_system_monitor() -> SystemMonitor:
    return system_monitor
```

-- 文件结束：monitor.py --

文件：build_rag.py （122 行，3.6 KB）

```python
# -*- coding: utf-8 -*-
"""
RAG 向量数据库构建脚本
"""
import os
import json
import time
import shutil
import chromadb
import requests
from config import (
    EMBED_API_URL, EMBED_MODEL, RAG_DB_DIR, RAG_COLLECTION_NAME,
    POEMS_JSON_DIR, EMBEDDING_RETRIES, EMBEDDING_TIMEOUT,
)
from errors import EmbeddingError
from logger import get_logger
logger = get_logger("build_rag")
POEMS_DIR = POEMS_JSON_DIR
def get_embedding(text, retries=EMBEDDING_RETRIES):
    """获取文本向量，带重试"""
    for attempt in range(retries):
        try:
            response = requests.post(
                EMBED_API_URL,
                json={"model": EMBED_MODEL, "input": text},
                timeout=EMBEDDING_TIMEOUT,
            )
            if response.status_code == 200:
                return response.json()["data"][0]["embedding"]
            else:
                logger.warning(f"API 异常：{response.status_code}，重试 {attempt+1}")
                time.sleep(2)
        except requests.RequestException as e:
            logger.warning(f"请求失败：{e}，重试 {attempt+1}")
            time.sleep(2)
    raise EmbeddingError(f"向量化失败，已重试 {retries} 次")
def main():
    """主构建流程"""
    if os.path.exists(RAG_DB_DIR):
        shutil.rmtree(RAG_DB_DIR)
        logger.info(f"已清除旧向量库：{RAG_DB_DIR}")
    logger.info(f"RAG 向量库构建启动")
    logger.info(f"读取目录：{POEMS_DIR}")
    logger.info(f"保存至：{RAG_DB_DIR}")
    client = chromadb.PersistentClient(path=RAG_DB_DIR)
    collection = client.get_or_create_collection(
        name=RAG_COLLECTION_NAME,
        metadata={"hnsw:space": "cosine"},
    )
    if not os.path.exists(POEMS_DIR):
        logger.error(f"数据目录不存在：{POEMS_DIR}")
        return
    all_files = [f for f in os.listdir(POEMS_DIR) if f.endswith(".json")]
    logger.info(f"共发现 {len(all_files)} 个文件")
    success_count = 0
    error_count = 0
    for filename in all_files:
        file_path = os.path.join(POEMS_DIR, filename)
        try:
            with open(file_path, "r", encoding="utf-8") as f:
                data = json.load(f)
        except Exception as e:
```

```
            logger.error(f"读取失败: {filename} | {e}")
            error_count += 1
            continue
        poems_in_file = data.get("诗歌集", [data]) if isinstance(data, dict) else data
        for poem in poems_in_file:
            pid = poem.get("诗歌编号", "")
            title = poem.get("标题", "未知")
            author = poem.get("作者", "未知")
            text = poem.get("原文", "")
            if not text or not pid:
                logger.warning(f"跳过空内容或无编号: {title}")
                continue
            logger.info(f"[{pid}] 《{title}》| {author} | {len(text)}字")
            try:
                vector = get_embedding(text)
            except EmbeddingError:
                logger.error(f"向量化失败，跳过: {pid}")
                error_count += 1
                continue
            try:
                collection.add(
                    ids=[pid],
                    embeddings=[vector],
                    documents=[text],
                    metadatas=[{"标题": title, "作者": author, "诗歌编号": pid}],
                )
                logger.info(f"入库成功: {pid}")
                success_count += 1
            except Exception as e:
                logger.error(f"入库失败: {pid} | {e}")
                error_count += 1
            time.sleep(0.3)
    logger.info(f"构建完成！成功: {success_count} | 失败: {error_count}")
if __name__ == "__main__":
    main()
```

-- 文件结束：build_rag.py --

文件：query_rag.py (238 行, 8.0 KB)

```
# -*- coding: utf-8 -*-
"""
RAG 检索与流式问答引擎
"""
import json
import chromadb
import requests
from config import (
    EMBED_API_URL, CHAT_API_URL, EMBED_MODEL, CHAT_MODEL,
    RAG_DB_DIR, RAG_COLLECTION_NAME, TOP_K, KNOWN_AUTHORS,
    EMBEDDING_RETRIES, EMBEDDING_TIMEOUT, CHAT_TIMEOUT,
)
from errors import EmbeddingError, LLMError
from logger import get_logger
logger = get_logger("query_rag")
def get_embedding(text):
    """获取文本向量"""
    for attempt in range(EMBEDDING_RETRIES):
        try:
            response = requests.post(
                EMBED_API_URL,
                json={"model": EMBED_MODEL, "input": text},
                timeout=EMBEDDING_TIMEOUT,
            )
            if response.status_code == 200:
                return response.json()["data"][0]["embedding"]
            else:
```

```
                logger.warning(f"Embedding API 异常: {response.status_code}，重试 {attempt+1}")
        except requests.RequestException as e:
            logger.warning(f"Embedding 请求失败: {e}，重试 {attempt+1}")
    raise EmbeddingError(f"向量化失败，已重试 {EMBEDDING_RETRIES} 次")
def extract_intent(question):
    """从问题中提取意图（检测诗人、清理无关词）"""
    detected_author = None
    for author in KNOWN_AUTHORS:
        if author in question:
            detected_author = author
            break
    clean_question = question
    if detected_author:
        clean_question = question.replace(detected_author, "").strip()
    for word in ["写过", "写了", "哪些", "有哪些", "的诗", "哪首", "什么诗", "请问", "帮我找"]:
        clean_question = clean_question.replace(word, "").strip()
    if not clean_question:
        clean_question = question
    return detected_author, clean_question
def search_poems(question, top_k=TOP_K):
    """在向量库中检索相关诗歌"""
    client = chromadb.PersistentClient(path=RAG_DB_DIR)
    collection = client.get_or_create_collection(
        name=RAG_COLLECTION_NAME,
        metadata={"hnsw:space": "cosine"},
    )
    detected_author, semantic_query = extract_intent(question)
    vector = get_embedding(semantic_query)
    if detected_author:
        try:
            results = collection.query(
                query_embeddings=[vector],
                n_results=top_k,
                where={"作者": detected_author},
            )
        except Exception:
            logger.warning(f"按作者过滤失败，回退到无过滤检索")
            results = collection.query(
                query_embeddings=[vector],
                n_results=top_k,
            )
    else:
        results = collection.query(
            query_embeddings=[vector],
            n_results=top_k,
        )
    poems = []
    for i in range(len(results["documents"][0])):
        poems.append({
            "原文": results["documents"][0][i],
            "标题": results["metadatas"][0][i].get("标题", ""),
            "作者": results["metadatas"][0][i].get("作者", ""),
            "相似度": round(1 - results["distances"][0][i], 3),
        })
    return poems
def _stream_chat_completion(messages, temperature=0.7, max_tokens=20000):
    """流式调用大模型"""
    try:
        response = requests.post(
            CHAT_API_URL,
            json={
                "model": CHAT_MODEL,
                "messages": messages,
                "temperature": temperature,
```

```
                "max_tokens": max_tokens,
                "stream": True,
            },
            timeout=CHAT_TIMEOUT,
            stream=True,
        )
        response.raise_for_status()
    except requests.RequestException as e:
        raise LLMError(f"大模型服务请求失败: {e}")
    for line in response.iter_lines():
        if not line:
            continue
        line_text = line.decode("utf-8")
        if line_text.startswith("data: "):
            line_text = line_text[6:]
        if line_text == "[DONE]":
            break
        try:
            chunk = json.loads(line_text)
            if not chunk.get("choices"):
                continue
            delta = chunk["choices"][0].get("delta", {})
            content = delta.get("content", "")
            if content:
                yield content
        except json.JSONDecodeError:
            continue
def build_context_block(poems):
    """构建检索到的诗歌上下文"""
    parts = []
    for p in poems:
        parts.append(f"《{p['标题']}》{p['作者']}\n{p['原文']}\n")
    return "\n".join(parts)
def _build_rag_messages(question, history):
    """构建 RAG 问答的消息列表"""
    history = history or []
    system = (
        "你是一位精通中国古典诗歌意象分析的学者，回答详尽准确，"
        "严格基于提供的资料，结合诗句原文进行分析。"
    )
    if not history:
        poems = search_poems(question)
        if not poems:
            return None, None
        ctx = build_context_block(poems)
        user_content = (
            f"你是一位精通中国古典诗歌的学者，请基于以下检索到的诗歌资料详细回答问题。\n"
            f"回答时请结合具体诗句分析，给出深入的文学解读。\n"
            f"注意：只基于提供的诗歌资料回答，不要编造不在资料中的诗歌。\n\n"
            f"诗歌资料：\n{ctx}\n\n问题：{question}\n\n请详细回答："
        )
        messages = [
            {"role": "system", "content": system},
            {"role": "user", "content": user_content},
        ]
        return messages, poems
    messages = [{"role": "system", "content": system}]
    for h in history:
        if h.get("role") in ("user", "assistant") and h.get("content"):
            messages.append({"role": h["role"], "content": h["content"]})
    messages.append({"role": "user", "content": question})
    return messages, None
def stream_rag_answer_events(question, history=None):
    """
```

```
    首轮：先 yield {"type":"poems","poems":[...]}，再 yield {"type":"chunk","text":"..."}
    后续轮：仅 yield chunk。无检索结果时仅 yield poems 空列表后结束。
    """
    messages, poems = _build_rag_messages(question, history)
    history = history or []
    if not history:
        yield {"type": "poems", "poems": poems or []}
        if not messages:
            return
    else:
        if not messages:
            return
    for c in _stream_chat_completion(messages):
        yield {"type": "chunk", "text": c}
def stream_rag_answer(question, history=None):
    """纯文本流，供命令行使用"""
    for ev in stream_rag_answer_events(question, history):
        if ev.get("type") == "chunk" and ev.get("text"):
            yield ev["text"]
def stream_cognitive_analysis(system_prompt, history=None, followup=None):
    """
    认知诗学意象解析：
    首轮：user 为 followup，缺省则使用默认解析指令。
    追问：history 为完整对话；followup 为新一轮用户输入。
    """
    history = history or []
    default_first_user = "请从认知诗学角度深度解析这个意象在诗歌中的多维功能。"
    if not history:
        user_msg = (followup or "").strip() or default_first_user
        messages = [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_msg},
        ]
    else:
        messages = [{"role": "system", "content": system_prompt}]
        for h in history:
            if h.get("role") in ("user", "assistant") and h.get("content"):
                messages.append({"role": h["role"], "content": h["content"]})
        q = (followup or "").strip() or "请继续从认知诗学角度补充分析。"
        messages.append({"role": "user", "content": q})
    yield from _stream_chat_completion(messages, temperature=0.65)
def ask(question):
    """命令行交互入口"""
    logger.info(f"问答: {question}")
    print(f"\n 问题：{question}")
    print("=" * 60)
    out = []
    for c in stream_rag_answer(question, None):
        out.append(c)
    print("".join(out))
    return "".join(out)
if __name__ == "__main__":
    ask("杜甫写过哪些关于秋天的诗？")
-- 文件结束：query_rag.py --
文件：cli.py  (240 行, 7.5 KB)
# -*- coding: utf-8 -*-
"""
CLI 命令行工具 — 系统运维与数据管理
"""
import argparse
import json
import os
import sys
import time
```

```
from config import BASE_DIR, POEM_JSON_DIR, RAG_DB_DIR, EXPORT_DIR
from logger import get_logger
logger = get_logger("cli")
def cmd_status(args):
    """显示系统状态"""
    from config import (
        EMBED_MODEL, CHAT_MODEL, FLASK_HOST, FLASK_PORT,
        CATEGORY_NAME_MAP, KNOWN_AUTHORS,
    )
    print("=" * 60)
    print("  唐诗意象智能分析系统 — 状态检查")
    print("=" * 60)
    print(f"\n  配置概览:")
    print(f"    Embedding 模型: {EMBED_MODEL}")
    print(f"    Chat 模型:      {CHAT_MODEL}")
    print(f"    服务地址:       http://{FLASK_HOST}:{FLASK_PORT}")
    print(f"    分类维度:       {len(CATEGORY_NAME_MAP)}")
    print(f"    已知诗人:       {len(KNOWN_AUTHORS)} 位")
    print(f"\n  目录状态:")
    for name, path in [("数据目录", POEM_JSON_DIR), ("向量库", RAG_DB_DIR),
                       ("导出目录", EXPORT_DIR)]:
        exists = os.path.exists(path)
        size = ""
        if exists:
            fcount = sum(1 for f in os.listdir(path)
                         if os.path.isfile(os.path.join(path, f)))
            size = f", {fcount} 个文件"
        print(f"    {name}: {'V' if exists else 'X'}{size}")
    print()
def cmd_scan(args):
    """扫描数据目录"""
    if not os.path.exists(POEM_JSON_DIR):
        print(f"数据目录不存在: {POEM_JSON_DIR}")
        return
    files = [f for f in os.listdir(POEM_JSON_DIR)
             if f.endswith((".json", ".txt"))]
    total_poems = 0
    total_units = 0
    print(f"\n扫描数据目录: {POEM_JSON_DIR}")
    print(f"找到 {len(files)} 个数据文件\n")
    for fname in sorted(files):
        fpath = os.path.join(POEM_JSON_DIR, fname)
        size_kb = os.path.getsize(fpath) / 1024
        try:
            with open(fpath, "r", encoding="utf-8") as f:
                data = json.load(f)
            poems = data.get("诗歌集", [data]) if isinstance(data, dict) else data
            pcount = len(poems)
            ucount = sum(len(p.get("分析单元", [])) for p in poems)
            total_poems += pcount
            total_units += ucount
            print(f"  {fname:40s} {size_kb:>8.1f} KB  {pcount:>3d} 首  {ucount:>4d} 单元")
        except Exception as e:
            print(f"  {fname:40s} {size_kb:>8.1f} KB  解析失败: {e}")
    print(f"\n总计: {total_poems} 首诗歌, {total_units} 个分析单元")
def cmd_export(args):
    """导出数据"""
    from app import get_data
    from export_service import ExportService
    data = get_data()
    traceback = data.get("traceback", [])
    fmt = args.format or "csv"
    if fmt == "csv":
        path = ExportService.export_traceback_to_csv(traceback)
```

```
    elif fmt == "json":
        path = ExportService.export_to_json(data)
    elif fmt == "report":
        from analytics import StatsService
        service = StatsService(traceback)
        path = ExportService.export_summary_report(service.summary_report())
    else:
        print(f"不支持格式: {fmt}")
        return
    fsize = os.path.getsize(path) / 1024
    print(f"导出成功: {path} ({fsize:.1f} KB)")
def cmd_clear_cache(args):
    """清理缓存"""
    from cache import memory_cache, file_cache
    memory_cache.clear()
    file_cache.clear()
    from app import clear_data_cache
    clear_data_cache()
    print("缓存已清理")
def cmd_list_exports(args):
    """列出导出文件"""
    from export_service import ExportService
    files = ExportService.list_exports()
    if not files:
        print("暂无导出文件")
        return
    print(f"\n导出目录: {EXPORT_DIR}")
    print(f"{'文件名':40s} {'大小':>10s}  {'修改时间'}")
    print("-" * 70)
    for f in files:
        size = f["size"] / 1024 if f["size"] > 1024 else f["size"]
        unit = "KB" if f["size"] > 1024 else "B"
        mtime = time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(f["mtime"]))
        print(f"  {f['name']:38s} {size:>8.2f} {unit}  {mtime}")
def cmd_check_rag(args):
    """检查向量库状态"""
    import chromadb
    if not os.path.exists(RAG_DB_DIR):
        print("向量库不存在，请先运行 build_rag.py")
        return
    client = chromadb.PersistentClient(path=RAG_DB_DIR)
    try:
        collection = client.get_collection(name="poems")
        count = collection.count()
        print(f"\n向量库状态:")
        print(f"  路径: {RAG_DB_DIR}")
        print(f"  集合: poems")
        print(f"  向量数: {count}")
        if count > 0:
            sample = collection.peek()
            print(f"  示例文档:")
            for i, doc in enumerate(sample["documents"][:3]):
                meta = sample["metadatas"][i]
                print(f"    [{i+1}] 《{meta.get('标题','')}》{meta.get('作者','')} ({len(doc)}字)")
    except Exception as e:
        print(f"读取向量库失败: {e}")
def cmd_build_rag(args):
    """构建向量库"""
    from build_rag import main as build_main
    build_main()
def cmd_test(args):
    """运行单元测试"""
    import subprocess
    test_path = os.path.join(BASE_DIR, "tests")
```

```
    cmd = [sys.executable, "-m", "pytest", test_path, "-v"]
    if args.verbose:
        cmd.append("-v")
    print(f"运行测试: {' '.join(cmd)}")
    result = subprocess.run(cmd, cwd=BASE_DIR)
    sys.exit(result.returncode)
def main():
    parser = argparse.ArgumentParser(
        description="唐诗意象智能分析系统 — CLI 管理工具",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="""
使用示例:
  python cli.py status          查看系统状态
  python cli.py scan            扫描数据目录
  python cli.py export --format csv   导出 CSV
  python cli.py test            运行单元测试
  python cli.py check-rag       检查向量库
  python cli.py build-rag       构建向量库
""",
    )
    subparsers = parser.add_subparsers(dest="command", help="可用命令")
    # status
    p = subparsers.add_parser("status", help="显示系统状态")
    p.set_defaults(func=cmd_status)
    # scan
    p = subparsers.add_parser("scan", help="扫描数据目录")
    p.set_defaults(func=cmd_scan)
    # export
    p = subparsers.add_parser("export", help="导出数据")
    p.add_argument("--format", "-f", choices=["csv", "json", "report"],
                   default="csv", help="导出格式")
    p.set_defaults(func=cmd_export)
    # clear-cache
    p = subparsers.add_parser("clear-cache", help="清理缓存")
    p.set_defaults(func=cmd_clear_cache)
    # list-exports
    p = subparsers.add_parser("list-exports", help="列出导出文件")
    p.set_defaults(func=cmd_list_exports)
    # check-rag
    p = subparsers.add_parser("check-rag", help="检查向量库状态")
    p.set_defaults(func=cmd_check_rag)
    # build-rag
    p = subparsers.add_parser("build-rag", help="构建向量库")
    p.set_defaults(func=cmd_build_rag)
    # test
    p = subparsers.add_parser("test", help="运行单元测试")
    p.add_argument("--verbose", "-v", action="store_true", help="详细输出")
    p.set_defaults(func=cmd_test)
    args = parser.parse_args()
    if args.command is None:
        parser.print_help()
        return
    try:
        args.func(args)
    except KeyboardInterrupt:
        print("\n操作已取消")
    except Exception as e:
        logger.exception("CLI 命令执行失败")
        print(f"错误: {e}")
        sys.exit(1)
if __name__ == "__main__":
    main()
```

-- 文件结束：cli.py --

文件：app.py （407 行, 15.9 KB）

```python
# -*- coding: utf-8 -*-
"""
唐诗意象智能分析系统 — Flask Web 主程序
"""
import json
import os
import glob
import re
from flask import Flask, render_template, request, Response
from config import (
    BASE_DIR, POEM_JSON_DIR, FLASK_HOST, FLASK_PORT, FLASK_DEBUG, FLASK_THREADED,
    SECRET_KEY, CATEGORY_NAME_MAP, PAGE_SIZE, RENDER_LIMIT,
)
from errors import register_error_handlers, AppError, ValidationError, sse_error
from validators import validate_question, validate_item, validate_history
from logger import get_logger, LoggerMixin
from cache import memory_cache, file_cache, cached
logger = get_logger("app")
app = Flask(__name__)
app.secret_key = SECRET_KEY
register_error_handlers(app)
from admin import admin_bp
app.register_blueprint(admin_bp)
# ═══════════════════════════════════════════════════════════════
# 数据加载
# ═══════════════════════════════════════════════════════════════
def clean_and_load_json(file_path):
    with open(file_path, "r", encoding="utf-8") as f:
        content = f.read().strip()
    content = re.sub(r"^```json\s*", "", content, flags=re.MULTILINE)
    content = re.sub(r"^```\s*", "", content, flags=re.MULTILINE)
    try:
        return json.loads(content)
    except json.JSONDecodeError:
        return None
def extract_poems(node):
    found_poems = []
    if isinstance(node, dict):
        if "分析单元" in node and isinstance(node.get("分析单元"), list):
            found_poems.append(node)
        else:
            for value in node.values():
                found_poems.extend(extract_poems(value))
    elif isinstance(node, list):
        for item in node:
            found_poems.extend(extract_poems(item))
    return found_poems
def build_traceback_dataset():
    """从 ./poem_json/ 扫描 JSON/TXT，生成溯源表与统计"""
    json_folder = POEM_JSON_DIR
    if not os.path.exists(json_folder):
        os.makedirs(json_folder, exist_ok=True)
    target_files = []
    for ext in ("*.json", "*.txt"):
        target_files.extend(glob.glob(os.path.join(json_folder, ext)))
    logger.info(f"扫描到 {len(target_files)} 个数据文件")
    traceback_data_list = []
    total_poems = 0
    seen_poem_fingerprints = set()
    skip_names = {
        "all_data.json", "dashboard_interactive_pro.html",
        "dashboard_FINAL_DASHBOARD.html", "Thesis_Dashboard.html",
    }
    for file_path in target_files:
```

```
        file_name = os.path.basename(file_path)
        if file_name in skip_names:
            continue
        data = clean_and_load_json(file_path)
        if not data:
            continue
        poems = extract_poems(data)
        if not poems:
            continue
        for p in poems:
            if not isinstance(p, dict):
                continue
            title = str(p.get("标题", "未知诗歌")).strip()
            poem_id = str(p.get("诗歌编号", p.get("编号", "-"))).strip()
            lines_data = p.get("诗行", [])
            first_line = ""
            if lines_data and isinstance(lines_data[0], dict):
                first_line = str(lines_data[0].get("原文", "")).strip()
            fingerprint = f"{title}_{first_line}"
            if fingerprint in seen_poem_fingerprints:
                continue
            seen_poem_fingerprints.add(fingerprint)
            total_poems += 1
            genre = str(p.get("分类标签", p.get("体裁", "未分类"))).strip()
            line_map = {}
            for l in lines_data:
                if isinstance(l, dict):
                    line_map[str(l.get("诗行编号", ""))] = l.get("原文", "")
            for u in p.get("分析单元", []):
                if not isinstance(u, dict):
                    continue
                if str(u.get("是否意象", "0")).strip() == "1":
                    text = str(u.get("文本", "")).strip()
                    sub_code = str(u.get("子类编码", "")).strip()
                    line_id = str(u.get("诗行编号", "")).strip()
                    line_text = line_map.get(line_id, "未知诗句")
                    cat_name = CATEGORY_NAME_MAP.get(sub_code, f"其他分类 ({sub_code})")
                    if text:
                        emo_cat = str(u.get("情感类别", "未知")).strip()
                        emo_pol = str(u.get("情感极性", "")).strip()
                        emotion_str = f"{emo_cat} ({emo_pol})" if emo_pol else emo_cat
                        dim_parts = []
                        for key in ["感知通道", "素材类型", "指涉来源", "表现功能"]:
                            val = str(u.get(key, "")).strip()
                            if val and val != "None":
                                dim_parts.append(val)
                        dimensions_str = " | ".join(dim_parts) if dim_parts else "-"
                        major_code = u.get("大类编码", "")
                        if major_code == "" or major_code is None:
                            major_code = (
                                sub_code.split("-")[0]
                                if sub_code and "-" in sub_code
                                else ""
                            )
                        traceback_data_list.append({
                            "poem_id": poem_id,
                            "title": title,
                            "author": p.get("作者", ""),
                            "genre": genre,
                            "cat": cat_name,
                            "txt": text,
                            "文本": text,
                            "dimensions": dimensions_str,
                            "emotion": emotion_str,
```

```
                            "emo_cat": emo_cat,
                            "line": line_text,
                            "id": line_id,
                            "词性": u.get("词性", ""),
                            "成分类型": u.get("成分类型", ""),
                            "感知通道": u.get("感知通道", ""),
                            "素材类型": u.get("素材类型", ""),
                            "内部结构": u.get("内部结构", ""),
                            "指涉来源": u.get("指涉来源", ""),
                            "表现功能": u.get("表现功能", ""),
                            "文化流通性": u.get("文化流通性", ""),
                            "跨文化性": u.get("跨文化性", ""),
                            "认知强度": u.get("认知强度", ""),
                            "核心意象": u.get("核心意象", ""),
                            "结构功能组": u.get("结构功能组", ""),
                            "情感极性": u.get("情感极性", ""),
                            "情感类别": u.get("情感类别", emo_cat),
                            "情感置信度": u.get("情感置信度", ""),
                            "大类编码": major_code,
                            "子类编码": sub_code,
                        })
    logger.info(f"数据加载完成: {total_poems} 首诗歌, {len(traceback_data_list)} 条意象")
    return {
        "total_poems": total_poems,
        "total_images": len(traceback_data_list),
        "traceback": traceback_data_list,
    }
_DATA_CACHE = None
def get_data():
    global _DATA_CACHE
    if _DATA_CACHE is None:
        _DATA_CACHE = build_traceback_dataset()
    return _DATA_CACHE
def clear_data_cache():
    global _DATA_CACHE
    _DATA_CACHE = None
    logger.info("数据缓存已清除")
# ──────────────────────────────────────────────────────────────────────
# SSE 工具函数
# ──────────────────────────────────────────────────────────────────────
def sse_json(obj):
    return f"data: {json.dumps(obj, ensure_ascii=False)}\n\n"
def build_analyze_system_prompt(item):
    """认知诗学解析：严格基于 JSON 结构化字段"""
    return f"""你是专精认知诗学与中国古典文学的学者。
当前意象数据：
- 意象文本：{item.get('文本', '')}
- 所属诗歌：{item.get('title', '')}
- 所在诗句：{item.get('line', '')}
- 词性：{item.get('词性', '')} | 成分类型：{item.get('成分类型', '')}
- 感知通道：{item.get('感知通道', '')} | 素材类型：{item.get('素材类型', '')}
- 内部结构：{item.get('内部结构', '')} | 指涉来源：{item.get('指涉来源', '')}
- 表现功能：{item.get('表现功能', '')} | 结构功能组：{item.get('结构功能组', '')}
- 文化流通性：{item.get('文化流通性', '')} | 跨文化性：{item.get('跨文化性', '')}
- 认知强度：{item.get('认知强度', '')} | 核心意象：{item.get('核心意象', '')}
- 情感极性：{item.get('情感极性', '')} | 情感类别：{item.get('情感类别', '')} | 置信度：{item.get('情感置信度', '')}
请按以下维度解析：
1. 感知层：触发的感官体验和通感效果
2. 文化层：典故来源和文化符号意义
3. 情感层：基于情感极性{item.get('情感极性', '')}分析情感功能
4. 结构层：作为{item.get('结构功能组', '')}在诗中的结构作用
5. 跨诗比较：此意象在其他诗歌中的异同
严格基于提供的数据分析，不编造。"""
# ──────────────────────────────────────────────────────────────────────
```

```
# 页面路由
# ──────────────────────────────────────────────────────────────────────
@app.route("/")
def index():
    return render_template("index.html")
# ──────────────────────────────────────────────────────────────────────
# 数据 API
# ──────────────────────────────────────────────────────────────────────
@app.route("/api/data")
def api_data():
    return Response(
        json.dumps(get_data(), ensure_ascii=False),
        mimetype="application/json; charset=utf-8",
    )
@app.route("/api/stats")
def api_stats_summary():
    """统计概览 API"""
    from analytics import StatsService, build_analytics_api
    data = get_data()
    traceback = data.get("traceback", [])
    service = StatsService(traceback)
    return Response(
        json.dumps(build_analytics_api(service), ensure_ascii=False),
        mimetype="application/json; charset=utf-8",
    )
@app.route("/api/export", methods=["POST"])
def api_export():
    """数据导出 API"""
    from export_service import ExportService
    payload = request.get_json(force=True, silent=True) or {}
    fmt = payload.get("format", "csv")
    data = get_data()
    traceback = data.get("traceback", [])
    try:
        if fmt == "csv":
            path = ExportService.export_traceback_to_csv(traceback)
        elif fmt == "json":
            path = ExportService.export_to_json(data)
        else:
            return Response(
                json.dumps({"error": f"不支持格式: {fmt}"}, ensure_ascii=False),
                mimetype="application/json; charset=utf-8",
                status=400,
            )
        return Response(
            json.dumps({"path": path, "file": os.path.basename(path)}, ensure_ascii=False),
            mimetype="application/json; charset=utf-8",
        )
    except Exception as e:
        logger.exception("导出失败")
        return Response(
            json.dumps({"error": str(e)}, ensure_ascii=False),
            mimetype="application/json; charset=utf-8",
            status=500,
        )
# ──────────────────────────────────────────────────────────────────────
# AI 问答 API（RAG）
# ──────────────────────────────────────────────────────────────────────
@app.route("/api/ask", methods=["POST"])
def api_ask():
    payload = request.get_json(force=True, silent=True) or {}
    question = (payload.get("question") or "").strip()
    history = validate_history(payload.get("history"))
    try:
```

```
        question = validate_question(question)
    except ValidationError as e:
        def empty():
            yield sse_error(e.message)
        return Response(empty(), mimetype="text/event-stream")
    from query_rag import stream_rag_answer_events
    def generate():
        try:
            for ev in stream_rag_answer_events(question, history):
                if ev.get("type") == "poems":
                    yield sse_json({"type": "poems", "poems": ev.get("poems") or []})
                    if not history and not (ev.get("poems") or []):
                        yield sse_json({
                            "type": "error",
                            "message": "未检索到相关诗歌，请调整提问或检查向量库。",
                        })
                        return
                elif ev.get("type") == "chunk" and ev.get("text"):
                    yield sse_json({"type": "chunk", "text": ev["text"]})
        except Exception as e:
            logger.exception("RAG 问答异常")
            yield sse_json({"type": "error", "message": str(e)})
    return Response(
        generate(),
        mimetype="text/event-stream",
        headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"},
    )
# ──────────────────────────────────────────────────────────────────────
# 认知诗学解析 API
# ──────────────────────────────────────────────────────────────────────
@app.route("/api/analyze", methods=["POST"])
def api_analyze():
    payload = request.get_json(force=True, silent=True) or {}
    item = payload.get("item") or {}
    history = validate_history(payload.get("history"))
    followup = payload.get("followup")
    try:
        item = validate_item(item)
    except ValidationError as e:
        def err():
            yield sse_error(e.message)
        return Response(err(), mimetype="text/event-stream")
    system_prompt = build_analyze_system_prompt(item)
    from query_rag import stream_cognitive_analysis
    def generate():
        try:
            for chunk in stream_cognitive_analysis(
                system_prompt,
                history=history,
                followup=followup,
            ):
                if chunk:
                    yield sse_json({"type": "chunk", "text": chunk})
        except Exception as e:
            logger.exception("认知解析异常")
            yield sse_json({"type": "error", "message": str(e)})
    return Response(
        generate(),
        mimetype="text/event-stream",
        headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"},
    )
# ──────────────────────────────────────────────────────────────────────
# 缓存管理 API
# ──────────────────────────────────────────────────────────────────────
```

```
@app.route("/api/cache/status")
def api_cache_status():
    return Response(
        json.dumps({
            "memory_entries": memory_cache.size(),
            "file_entries": file_cache.size(),
            "data_cache_active": _DATA_CACHE is not None,
        }),
        mimetype="application/json; charset=utf-8",
    )
# ──────────────────────────────────────────────────────────────────────────────
# 启动入口
# ──────────────────────────────────────────────────────────────────────────────
if __name__ == "__main__":
    logger.info(f"启动唐诗意象智能分析系统 (http://{FLASK_HOST}:{FLASK_PORT})")
    app.run(host=FLASK_HOST, port=FLASK_PORT, debug=FLASK_DEBUG, threaded=FLASK_THREADED)
```

-- 文件结束：app.py --

文件：admin.py （176 行, 5.1 KB）

```
# -*- coding: utf-8 -*-
"""
管理后台 Blueprint — 系统状态、数据管理、导出操作
"""
import json
import os
import time
from flask import Blueprint, render_template, jsonify, request, Response
from config import (
    BASE_DIR, RAG_DB_DIR, POEM_JSON_DIR, EXPORT_DIR,
    EMBED_MODEL, CHAT_MODEL, FLASK_PORT,
    CATEGORY_NAME_MAP, KNOWN_AUTHORS, PAGE_SIZE, TOP_K,
)
from analytics import StatsService
from cache import memory_cache, file_cache
from export_service import ExportService
from logger import get_logger
logger = get_logger("admin")
admin_bp = Blueprint("admin", __name__, url_prefix="/admin")
def _get_data_provider():
    """延迟导入以避免循环引用"""
    from app import get_data
    return get_data()
@admin_bp.route("/")
def admin_index():
    return render_template("admin.html")
@admin_bp.route("/api/status")
def api_status():
    """系统状态总览"""
    data = _get_data_provider()
    total_images = len(data.get("traceback", []))
    total_poems = data.get("total_poems", 0)
    rag_exists = os.path.exists(RAG_DB_DIR) and os.path.isdir(RAG_DB_DIR)
    rag_file_count = 0
    if rag_exists:
        for root, dirs, files in os.walk(RAG_DB_DIR):
            rag_file_count += len(files)
    chunk_count = 0
    if os.path.exists(POEM_JSON_DIR):
        chunk_count = len([f for f in os.listdir(POEM_JSON_DIR)
                           if f.endswith((".json", ".txt"))])
    cache_info = {
        "memory_entries": memory_cache.size(),
        "file_entries": file_cache.size(),
    }
    export_files = ExportService.list_exports()
```

```
    return jsonify({
        "status": "running",
        "timestamp": time.time(),
        "data": {
            "total_poems": total_poems,
            "total_images": total_images,
            "category_count": len(CATEGORY_NAME_MAP),
            "known_authors_count": len(KNOWN_AUTHORS),
        },
        "rag_database": {
            "exists": rag_exists,
            "file_count": rag_file_count,
            "path": RAG_DB_DIR,
        },
        "poem_json": {
            "file_count": chunk_count,
            "path": POEM_JSON_DIR,
        },
        "cache": cache_info,
        "exports": {
            "file_count": len(export_files),
            "directory": EXPORT_DIR,
        },
        "config": {
            "embed_model": EMBED_MODEL,
            "chat_model": CHAT_MODEL,
            "port": FLASK_PORT,
            "top_k": TOP_K,
            "page_size": PAGE_SIZE,
        },
    })
@admin_bp.route("/api/stats")
def api_stats():
    """统计分析数据"""
    data = _get_data_provider()
    traceback = data.get("traceback", [])
    service = StatsService(traceback)
    report = service.summary_report()
    report["cross_analysis"] = service.cross_analysis()
    report["author_stats"] = service.author_statistics()
    report["perception_stats"] = service.perception_channel_distribution()
    return jsonify(report)
@admin_bp.route("/api/exports", methods=["GET"])
def api_list_exports():
    """列出导出文件"""
    return jsonify({"exports": ExportService.list_exports()})
@admin_bp.route("/api/exports", methods=["POST"])
def api_create_export():
    """创建导出"""
    payload = request.get_json(force=True, silent=True) or {}
    fmt = payload.get("format", "csv")
    data = _get_data_provider()
    traceback = data.get("traceback", [])
    try:
        if fmt == "csv":
            path = ExportService.export_traceback_to_csv(traceback)
        elif fmt == "json":
            path = ExportService.export_to_json(data)
        elif fmt == "report":
            service = StatsService(traceback)
            path = ExportService.export_summary_report(service.summary_report())
        else:
            return jsonify({"error": f"不支持的导出格式: {fmt}"}), 400
        fname = os.path.basename(path)
```

```
        fsize = os.path.getsize(path)
        return jsonify({"message": "导出成功", "file": fname, "size": fsize, "path": path})
    except Exception as e:
        logger.exception("导出失败")
        return jsonify({"error": str(e)}), 500
@admin_bp.route("/api/exports/clear", methods=["POST"])
def api_clear_exports():
    """清空导出目录"""
    count = ExportService.clear_exports()
    return jsonify({"message": f"已移除 {count} 个导出文件", "count": count})
@admin_bp.route("/api/cache/clear", methods=["POST"])
def api_clear_cache():
    """清理缓存"""
    cache_type = (request.get_json(force=True, silent=True) or {}).get("type", "all")
    if cache_type in ("memory", "all"):
        memory_cache.clear()
    if cache_type in ("file", "all"):
        file_cache.clear()
    from app import clear_data_cache
    clear_data_cache()
    logger.info(f"缓存已清理 (type={cache_type})")
    return jsonify({"message": "缓存已清理", "type": cache_type})
@admin_bp.route("/api/refresh", methods=["POST"])
def api_refresh_data():
    """刷新数据缓存"""
    from app import clear_data_cache, get_data
    clear_data_cache()
    data = get_data()
    return jsonify({
        "message": "数据已刷新",
        "total_poems": data.get("total_poems", 0),
        "total_images": len(data.get("traceback", [])),
    })
```

-- 文件结束：admin.py --

文件：templates/index.html (1547 行, 55.1 KB)

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>唐诗意象智能分析系统</title>
  <script src="https://cdn.jsdelivr.net/npm/echarts@5.5.0/dist/echarts.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/marked/marked.min.js"></script>
  <style>
    :root {
      --primary: #2c3e50;
      --accent: #3498db;
      --bg: #f4f7f6;
      --text: #333;
      --card: #ffffff;
      --success: #2ecc71;
      --warn: #e67e22;
    }
    * { box-sizing: border-box; }
    body {
      font-family: 'Helvetica Neue', Arial, sans-serif;
      background-color: var(--bg);
      margin: 0;
      padding: 0;
      color: var(--text);
      padding-top: 64px;
    }
    .navbar {
      position: fixed;
```

```
      top: 0;
      left: 0;
      right: 0;
      height: 64px;
      background: var(--card);
      border-bottom: 1px solid #e0e0e0;
      box-shadow: 0 2px 10px rgba(0,0,0,0.04);
      display: flex;
      align-items: center;
      justify-content: space-between;
      padding: 0 24px;
      z-index: 900;
    }
    .brand {
      display: flex;
      align-items: center;
      gap: 12px;
      font-size: 18px;
      font-weight: 700;
      color: var(--primary);
    }
    .logo {
      width: 36px;
      height: 36px;
      border-radius: 8px;
      background: linear-gradient(135deg, var(--accent), #2980b9);
      display: flex;
      align-items: center;
      justify-content: center;
      color: #fff;
      font-size: 14px;
      font-weight: 800;
    }
    .tabs {
      display: flex;
      gap: 8px;
    }
    .tab-btn {
      border: 1px solid #dce4ec;
      background: #fff;
      color: var(--primary);
      padding: 10px 22px;
      border-radius: 8px;
      cursor: pointer;
      font-weight: 600;
      transition: background 0.25s, color 0.25s, border-color 0.25s, transform 0.15s;
    }
    .tab-btn:hover { border-color: var(--accent); color: var(--accent); transform: translateY(-1px); }
    .tab-btn.active {
      background: var(--accent);
      color: #fff;
      border-color: var(--accent);
    }
    .tab-pane {
      display: none;
      opacity: 0;
      transition: opacity 0.35s ease;
      max-width: 1600px;
      margin: 24px auto;
      padding: 0 20px 40px;
    }
    .tab-pane.visible {
      display: block;
      opacity: 1;
```

```
    }
    .stats-bar-wrap {
      text-align: center;
      margin-bottom: 24px;
    }
    .stats-bar {
      display: inline-flex;
      flex-wrap: wrap;
      gap: 24px;
      justify-content: center;
      background: #f8f9fa;
      padding: 14px 28px;
      border-radius: 50px;
      border: 1px solid #eee;
    }
    .stat-item { font-size: 15px; font-weight: bold; color: #555; }
    .stat-num { color: var(--accent); font-size: 19px; margin-left: 6px; }
    .card {
      background: var(--card);
      border-radius: 10px;
      padding: 24px;
      margin-bottom: 24px;
      box-shadow: 0 4px 20px rgba(0,0,0,0.04);
      transition: 0.25s;
      position: relative;
    }
    .card-header {
      display: flex;
      justify-content: space-between;
      align-items: center;
      margin-bottom: 16px;
      border-bottom: 2px solid #f0f0f0;
      padding-bottom: 12px;
      flex-wrap: wrap;
      gap: 12px;
    }
    .card-title {
      margin: 0;
      font-size: 19px;
      color: var(--primary);
      border-left: 5px solid var(--accent);
      padding-left: 12px;
    }
    .btn-action {
      background: #ecf0f1;
      color: var(--primary);
      border: none;
      padding: 8px 16px;
      border-radius: 8px;
      cursor: pointer;
      font-weight: bold;
      transition: 0.2s;
      display: inline-flex;
      align-items: center;
      gap: 6px;
    }
    .btn-action:hover { background: var(--accent); color: #fff; }
    .grid-2 {
      display: grid;
      grid-template-columns: 1fr 1fr;
      gap: 24px;
    }
    @media (max-width: 960px) {
      .grid-2 { grid-template-columns: 1fr; }
```

```
    .navbar { flex-wrap: wrap; height: auto; padding: 12px; gap: 12px; }
    body { padding-top: 100px; }
  }
  .chart-container { height: 400px; width: 100%; }
  .filter-bar {
    display: flex;
    gap: 12px;
    margin-bottom: 16px;
    flex-wrap: wrap;
    background: #f8f9fa;
    padding: 14px;
    border-radius: 8px;
  }
  .filter-input, .filter-select {
    padding: 10px 14px;
    border: 1px solid #ddd;
    border-radius: 8px;
    font-size: 14px;
    outline: none;
    transition: 0.25s;
  }
  .filter-input:focus, .filter-select:focus {
    border-color: var(--accent);
    box-shadow: 0 0 8px rgba(52, 152, 219, 0.35);
  }
  .filter-input { flex: 1; min-width: 200px; }
  .table-wrapper {
    max-height: 520px;
    overflow: auto;
    border: 1px solid #eee;
    border-radius: 8px;
  }
  table {
    width: 100%;
    border-collapse: collapse;
    font-size: 14px;
    table-layout: fixed;
  }
  th, td {
    border: 1px solid #eee;
    padding: 10px 12px;
    text-align: left;
    vertical-align: middle;
    word-wrap: break-word;
    line-height: 1.5;
  }
  th {
    background: #fafafa;
    position: sticky;
    top: 0;
    z-index: 5;
    font-weight: 600;
  }
  tbody tr.data-row {
    transition: background 0.2s;
    cursor: default;
  }
  tbody tr.data-row:hover { background-color: #eaf4fc; }
  .tag {
    background: #ecf0f1;
    padding: 3px 8px;
    border-radius: 4px;
    font-size: 12px;
    color: #555;
```

```
    display: inline-block;
  }
  .recycle-btn {
    background: var(--warn);
    color: #fff;
    padding: 10px 18px;
    border-radius: 30px;
    border: none;
    font-weight: bold;
    cursor: pointer;
    transition: 0.25s;
    box-shadow: 0 4px 10px rgba(230, 126, 34, 0.25);
  }
  .recycle-btn:hover { filter: brightness(1.05); transform: translateY(-2px); }
  .pager {
    display: flex;
    align-items: center;
    gap: 12px;
    justify-content: flex-end;
    margin-top: 12px;
    flex-wrap: wrap;
  }
  .pager button {
    padding: 8px 14px;
    border-radius: 8px;
    border: 1px solid #dce4ec;
    background: #fff;
    cursor: pointer;
    transition: 0.2s;
  }
  .pager button:hover:not(:disabled) { border-color: var(--accent); color: var(--accent); }
  .pager button:disabled { opacity: 0.45; cursor: not-allowed; }
  .modal-overlay {
    display: none;
    position: fixed;
    inset: 0;
    background: rgba(0,0,0,0.55);
    z-index: 950;
    backdrop-filter: blur(3px);
  }
  .modal-content {
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);
    background: #fff;
    width: min(1000px, 92vw);
    max-height: 82vh;
    border-radius: 12px;
    padding: 28px;
    box-shadow: 0 10px 30px rgba(0,0,0,0.2);
    display: flex;
    flex-direction: column;
  }
  .close-btn {
    position: absolute;
    top: 16px;
    right: 16px;
    background: none;
    border: none;
    font-size: 22px;
    cursor: pointer;
    color: #999;
  }
```

```
.close-btn:hover { color: #e74c3c; }
/* 意象解析 — 右侧抽屉 */
.drawer-overlay {
  position: fixed;
  inset: 0;
  background: rgba(0, 0, 0, 0.45);
  z-index: 1000;
  opacity: 0;
  visibility: hidden;
  transition: opacity 0.3s ease, visibility 0.3s ease;
}
.drawer-overlay.show {
  opacity: 1;
  visibility: visible;
}
.analyze-drawer {
  position: fixed;
  top: 0;
  right: 0;
  width: 420px;
  max-width: 100vw;
  height: 100vh;
  background: #fff;
  z-index: 1001;
  box-shadow: -4px 0 20px rgba(0, 0, 0, 0.1);
  transform: translateX(100%);
  transition: transform 0.3s ease;
  display: flex;
  flex-direction: column;
}
.analyze-drawer.open {
  transform: translateX(0);
}
/* 抽屉拖拽边框 */
.drawer-resizer {
  position: absolute;
  left: 0;
  top: 0;
  bottom: 0;
  width: 6px;
  cursor: ew-resize;
  z-index: 1005;
  background: transparent;
  transition: background 0.2s;
}
.drawer-resizer:hover, .drawer-resizer.active {
  background: rgba(52, 152, 219, 0.4);
}
.analyze-drawer.no-transition {
  transition: none !important;
}
.drawer-head {
  flex-shrink: 0;
  padding: 16px 44px 14px 18px;
  border-bottom: 1px solid #eee;
  position: relative;
}
.drawer-close {
  position: absolute;
  top: 12px;
  right: 12px;
  width: 36px;
  height: 36px;
  border: none;
```

```
    background: transparent;
    font-size: 22px;
    line-height: 1;
    cursor: pointer;
    color: #999;
    border-radius: 8px;
  }
  .drawer-close:hover { color: #e74c3c; background: #f5f5f5; }
  .drawer-title-row {
    font-size: 16px;
    font-weight: 700;
    color: var(--primary);
    margin-bottom: 8px;
  }
  .drawer-title-row .hl { color: var(--accent); }
  .drawer-meta {
    font-size: 13px;
    color: #666;
    margin-bottom: 6px;
  }
  .drawer-line {
    font-family: 楷体, KaiTi, serif;
    font-size: 15px;
    line-height: 1.55;
    color: #333;
  }
  .drawer-tags {
    flex-shrink: 0;
    padding: 10px 14px;
    border-bottom: 1px solid #eee;
    background: #fafbfc;
    max-height: 120px;
    overflow-y: auto;
  }
  .tag-chip {
    display: inline-block;
    padding: 3px 10px;
    border-radius: 20px;
    background: #eaf4fd;
    color: #3498db;
    font-size: 12px;
    margin: 3px;
  }
  .drawer-chat-scroll {
    flex: 1;
    overflow-y: auto;
    overflow-x: hidden;
    padding: 14px 16px;
    min-height: 0;
  }
  .msg-row {
    display: flex;
    margin-bottom: 12px;
  }
  .msg-row.user { justify-content: flex-end; }
  .msg-row.assistant { justify-content: flex-start; }
  .msg-bubble {
    max-width: 92%;
    padding: 10px 12px;
    border-radius: 14px;
    font-size: 14px;
    line-height: 1.65;
    word-break: break-word;
  }
```

```
    .msg-bubble.user {
      background: #3498db;
      color: #fff;
      border-bottom-right-radius: 4px;
    }
    .msg-bubble.assistant {
      background: #ecf0f1;
      color: #333;
      border-bottom-left-radius: 4px;
    }
    .drawer-md { font-size: 14px; line-height: 1.65; }
    .drawer-md h1, .drawer-md h2, .drawer-md h3 { margin: 0.6em 0 0.35em; font-size: 1.05em; }
    .drawer-md p { margin: 0.45em 0; }
    .drawer-md ul, .drawer-md ol { margin: 0.4em 0; padding-left: 1.2em; }
    .drawer-md code { background: rgba(0,0,0,0.06); padding: 2px 5px; border-radius: 4px; font-size: 0.9em; }
    .drawer-foot {
      flex-shrink: 0;
      padding: 12px 14px;
      border-top: 1px solid #eee;
      background: #fafafa;
      display: flex;
      gap: 10px;
      align-items: center;
    }
    .drawer-foot input {
      flex: 1;
      padding: 11px 12px;
      border-radius: 8px;
      border: 1px solid #ddd;
      outline: none;
      font-size: 14px;
    }
    .drawer-foot input:focus {
      border-color: var(--accent);
      box-shadow: 0 0 8px rgba(52, 152, 219, 0.35);
    }
    .drawer-foot button {
      padding: 11px 18px;
      border-radius: 8px;
      border: none;
      background: var(--accent);
      color: #fff;
      font-weight: 600;
      cursor: pointer;
      white-space: nowrap;
    }
    .drawer-foot button:hover { filter: brightness(1.05); }
    /* AI tab */
    .ai-hero {
      max-width: 720px;
      margin: 0 auto 28px;
      text-align: center;
    }
    .ai-hero input {
      width: 100%;
      padding: 16px 20px;
      font-size: 17px;
      border-radius: 12px;
      border: 1px solid #ddd;
      outline: none;
      transition: 0.25s;
    }
    .ai-hero input:focus {
      border-color: var(--accent);
```

```
    box-shadow: 0 0 12px rgba(52, 152, 219, 0.35);
  }
  .ai-hero button {
    margin-top: 14px;
    padding: 12px 32px;
    border-radius: 10px;
    border: none;
    background: var(--accent);
    color: #fff;
    font-size: 16px;
    font-weight: 700;
    cursor: pointer;
    transition: 0.2s;
  }
  .ai-hero button:hover { filter: brightness(1.05); transform: translateY(-1px); }
  .ai-split {
    display: none;
    grid-template-columns: 40% 60%;
    gap: 20px;
    align-items: stretch;
    min-height: 60vh;
  }
  .ai-split.active { display: grid; }
  @media (max-width: 900px) {
    .ai-split.active { grid-template-columns: 1fr; }
  }
  .poem-col {
    overflow: auto;
    max-height: calc(100vh - 220px);
  }
  .poem-card {
    background: var(--card);
    border-radius: 10px;
    padding: 14px 16px;
    margin-bottom: 12px;
    box-shadow: 0 4px 20px rgba(0,0,0,0.04);
    cursor: pointer;
    transition: transform 0.2s, box-shadow 0.2s;
    border: 1px solid #f0f0f0;
  }
  .poem-card:hover {
    transform: translateY(-3px);
    box-shadow: 0 8px 24px rgba(0,0,0,0.08);
  }
  .poem-card h4 { margin: 0 0 6px; color: var(--primary); font-size: 15px; }
  .poem-card .sub { font-size: 12px; color: #888; margin-bottom: 8px; }
  .poem-card .body {
    font-family: 楷体, KaiTi, serif;
    font-size: 15px;
    line-height: 1.65;
    color: #333;
  }
  .poem-card .sim {
    margin-top: 8px;
    font-size: 12px;
    color: var(--success);
    font-weight: 700;
  }
  .poem-detail { display: none; margin-top: 10px; font-size: 13px; color: #555; white-space: pre-wrap; }
  .poem-card.expanded .poem-detail { display: block; }
  .ai-answer {
    background: var(--card);
    border-radius: 10px;
    padding: 18px;
```

```
      box-shadow: 0 4px 20px rgba(0,0,0,0.04);
      display: flex;
      flex-direction: column;
      min-height: 420px;
      border: 1px solid #f0f0f0;
    }
    .ai-answer-scroll {
      flex: 1;
      overflow: auto;
      padding-bottom: 12px;
      white-space: pre-wrap;
      line-height: 1.75;
      font-size: 15px;
    }
    .ai-foot {
      margin-top: 10px;
      border-top: 1px solid #eee;
      padding-top: 12px;
      display: flex;
      gap: 10px;
    }
    .ai-foot input {
      flex: 1;
      padding: 12px 14px;
      border-radius: 8px;
      border: 1px solid #ddd;
      outline: none;
    }
    .ai-foot input:focus {
      border-color: var(--accent);
      box-shadow: 0 0 8px rgba(52, 152, 219, 0.35);
    }
    .ai-foot button {
      padding: 12px 18px;
      border-radius: 8px;
      border: none;
      background: var(--primary);
      color: #fff;
      font-weight: 600;
      cursor: pointer;
    }
    .warn-text { color: var(--warn); font-size: 13px; margin-top: 8px; }
    .link-action { color: var(--accent); cursor: pointer; font-weight: 600; }
    .danger { color: #e74c3c; }
    .ops-cell { white-space: nowrap; }
    .ops-cell .btn-link {
      background: none;
      border: none;
      padding: 0 8px;
      cursor: pointer;
      font-size: 14px;
      color: var(--accent);
      font-weight: 600;
    }
    .ops-cell .btn-link:hover { text-decoration: underline; }
  </style>
</head>
<body>
  <nav class="navbar">
    <div class="brand">
      <div class="logo">唐</div>
      <span>唐诗意象智能分析系统</span>
    </div>
    <div class="tabs">
```

```
      <button type="button" class="tab-btn active" data-tab="graph">数据图谱</button>
      <button type="button" class="tab-btn" data-tab="ai">AI问答</button>
    </div>
  </nav>
  <section id="tab-graph" class="tab-pane visible">
    <div class="stats-bar-wrap">
      <div class="stats-bar">
        <div class="stat-item">净解析诗歌：<span id="stat-poems" class="stat-num">0</span> 首</div>
        <div class="stat-item">提取真意象：<span id="stat-images" class="stat-num">0</span> 条</div>
        <div class="stat-item">分类维度：<span class="stat-num">4 层 10 维</span></div>
      </div>
      <div style="margin-top:16px;">
        <button type="button" class="recycle-btn" onclick="openRecycle()">回收站 (<span id="recycle-count">0</span>)</button>
      </div>
    </div>
    <div class="grid-2">
      <div class="card" id="card-ind">
        <div class="card-header">
          <h2 class="card-title">核心意象 Top50</h2>
          <button type="button" class="btn-action" onclick="toggleFullScreen('card-ind')">放大视图</button>
        </div>
        <div id="chart-ind" class="chart-container"></div>
      </div>
      <div class="card" id="card-cat">
        <div class="card-header">
          <h2 class="card-title">意象分类域分布</h2>
          <button type="button" class="btn-action" onclick="toggleFullScreen('card-cat')">放大视图</button>
        </div>
        <div id="chart-cat" class="chart-container"></div>
      </div>
    </div>
    <div class="card">
      <div class="card-header">
        <h2 class="card-title">意象溯源数据表 <span id="record-count" style="font-size:13px;color:#999;font-weight:400;"></span></h2>
        <button type="button" class="btn-action" style="background:#e74c3c;color:#fff;display:none;" id="resetBtn" onclick="resetFilters()">清除
      </div>
      <div class="filter-bar">
        <input type="text" id="searchTxt" class="filter-input" placeholder="搜索意象文本（如：山、月）..." onkeyup="applyFilters()">
        <select id="filterCat" class="filter-select" onchange="applyFilters()">
          <option value="">所有分类域</option>
        </select>
        <select id="filterGenre" class="filter-select" onchange="applyFilters()">
          <option value="">所有体裁/标签</option>
        </select>
        <select id="filterEmotion" class="filter-select" onchange="applyFilters()">
          <option value="">所有情感倾向</option>
        </select>
      </div>
      <div class="table-wrapper">
        <table id="dataTable">
          <thead>
            <tr>
              <th style="width:9%;">诗歌编号</th>
              <th style="width:11%;">意象文本</th>
              <th style="width:20%;">所属诗歌</th>
              <th style="width:14%;">体裁/标签</th>
              <th style="width:18%;">大类归属</th>
              <th style="width:12%;">情感倾向</th>
              <th style="width:14%;">操作</th>
            </tr>
          </thead>
          <tbody id="traceback-body"></tbody>
        </table>
      </div>
```

```
      <div class="pager">
        <span id="page-info"></span>
        <button type="button" id="page-prev" onclick="changePage(-1)">上一页</button>
        <button type="button" id="page-next" onclick="changePage(1)">下一页</button>
      </div>
    </div>
  </section>
  <section id="tab-ai" class="tab-pane">
    <div class="ai-hero">
      <input type="text" id="rag-question" placeholder="输入关于唐诗的问题，例如：杜甫写过哪些关于秋天的诗？" onkeydown="if(event.key==='Enter')
      <div>
        <button type="button" onclick="runRagFirst()">检索并生成回答</button>
      </div>
      <p class="warn-text" id="rag-hint" style="display:none;"></p>
    </div>
    <div id="ai-split" class="ai-split">
      <div class="poem-col" id="rag-poems"></div>
      <div class="ai-answer">
        <div class="ai-answer-scroll" id="rag-answer"></div>
        <div class="ai-foot">
          <input type="text" id="rag-followup" placeholder="多轮追问..." onkeydown="if(event.key==='Enter'){runRagFollowup();}">
          <button type="button" onclick="runRagFollowup()">发送</button>
        </div>
      </div>
    </div>
  </section>
  <div class="modal-overlay" id="detailModal">
    <div class="modal-content">
      <button type="button" class="close-btn" onclick="closeModal('detailModal')">×</button>
      <div id="md-content"></div>
    </div>
  </div>
  <div id="drawer-overlay" class="drawer-overlay" onclick="closeAnalyzeDrawer()"></div>
  <aside id="analyze-drawer" class="analyze-drawer" aria-hidden="true">
    <div id="drawer-resizer" class="drawer-resizer"></div>
    <div class="drawer-head">
      <button type="button" class="drawer-close" onclick="closeAnalyzeDrawer()" aria-label="关闭">×</button>
      <div class="drawer-title-row" id="drawer-title-line">意象：<span class="hl" id="drawer-img-text">—</span></div>
      <div class="drawer-meta" id="drawer-poem-meta">《》</div>
      <div class="drawer-line" id="drawer-line-text">诗句：—</div>
    </div>
    <div class="drawer-tags" id="drawer-tags"></div>
    <div class="drawer-chat-scroll" id="drawer-chat-scroll">
      <div id="drawer-messages"></div>
    </div>
    <div class="drawer-foot">
      <input type="text" id="drawer-followup" placeholder="输入追问…" onkeydown="if(event.key==='Enter'){sendDrawerFollowup();}">
      <button type="button" onclick="sendDrawerFollowup()">发送</button>
    </div>
  </aside>
  <div class="modal-overlay" id="recycleModal">
    <div class="modal-content">
      <button type="button" class="close-btn" onclick="closeModal('recycleModal')">×</button>
      <h2 style="color:var(--warn);margin-top:0;border-bottom:2px solid #eee;padding-bottom:12px;">已删除意象回收站</h2>
      <div class="table-wrapper" style="flex:1;max-height:62vh;">
        <table>
          <thead>
            <tr>
              <th style="width:12%;">编号</th>
              <th style="width:15%;">被删意象</th>
              <th style="width:25%;">所属诗歌</th>
              <th style="width:28%;">大类</th>
              <th style="width:18%;">操作</th>
            </tr>
```

```
          </thead>
          <tbody id="recycle-body"></tbody>
        </table>
      </div>
    </div>
  </div>
  <script>
    const PAGE_SIZE = 25;
    const RENDER_LIMIT = 5000;
    let ALL_DATA = [];
    let DELETED_DATA = [];
    let filteredView = [];
    let currentPage = 1;
    let totalPoemsStat = 0;
    let indChart = null;
    let catChart = null;
    const DEFAULT_ANALYZE_PROMPT = '请从认知诗学角度深度解析这个意象在诗歌中的多维功能';
    /** 意象解析缓存 key = poem_id + '_' + 意象文本 */
    const ANALYZE_CACHE = {};
    let currentCacheKey = null;
    function makeCacheKey(item) {
      if (!item) return '';
      const pid = String(item.poem_id ?? '');
      const tx = String(item.txt ?? item['文本'] ?? '');
      return pid + '_' + tx;
    }
    function isCacheAnalysisComplete(ent) {
      if (!ent) return false;
      if (ent.done === true) return true;
      if (ent.done === false) return false;
      return Array.isArray(ent.messages) && ent.messages.length >= 2;
    }
    let ragPoems = [];
    let ragConv = [];
    let ragBusy = false;
    function ctxHint(item) {
      const t = (item && (item['文本'] || item.txt)) || '';
      return `（语境：《${(item && item.title) || ''}》| 意象：${t} | 诗句：${(item && item.line) || ''}）\n`;
    }
    function itemApiPayload(row) {
      if (!row) return {};
      const o = JSON.parse(JSON.stringify(row));
      delete o._uid;
      return o;
    }
    async function loadApiData() {
      const res = await fetch('/api/data');
      const data = await res.json();
      totalPoemsStat = data.total_poems || 0;
      ALL_DATA = data.traceback || [];
      document.getElementById('stat-poems').innerText = totalPoemsStat;
      ALL_DATA.forEach((item, index) => {
        item._uid = 'img_' + index + '_' + Math.random().toString(36).substr(2, 5);
      });
      initCharts();
      refreshAll();
    }
    function initCharts() {
      const el1 = document.getElementById('chart-ind');
      const el2 = document.getElementById('chart-cat');
      if (!indChart) indChart = echarts.init(el1);
      if (!catChart) catChart = echarts.init(el2);
    }
    function updateUIStats() {
```

```
    document.getElementById('stat-images').innerText = ALL_DATA.length;
    document.getElementById('recycle-count').innerText = DELETED_DATA.length;
  }
  function populateSelects() {
    const cats = [...new Set(ALL_DATA.map(item => item.cat))].filter(Boolean);
    const genres = [...new Set(ALL_DATA.map(item => item.genre))].filter(Boolean);
    const emotions = [...new Set(ALL_DATA.map(item => item.emo_cat))].filter(Boolean);
    const catSel = document.getElementById('filterCat');
    const genSel = document.getElementById('filterGenre');
    const emoSel = document.getElementById('filterEmotion');
    const curCat = catSel.value, curGen = genSel.value, curEmo = emoSel.value;
    catSel.innerHTML = '<option value="">所有分类域</option>';
    genSel.innerHTML = '<option value="">所有体裁/标签</option>';
    emoSel.innerHTML = '<option value="">所有情感倾向</option>';
    cats.forEach(c => catSel.add(new Option(c, c)));
    genres.forEach(g => genSel.add(new Option(g, g)));
    emotions.forEach(e => emoSel.add(new Option(e, e)));
    catSel.value = curCat; genSel.value = curGen; emoSel.value = curEmo;
  }
  function getFiltered() {
    const searchTxt = document.getElementById('searchTxt').value.toLowerCase();
    const selCat = document.getElementById('filterCat').value;
    const selGen = document.getElementById('filterGenre').value;
    const selEmo = document.getElementById('filterEmotion').value;
    return ALL_DATA.filter(item => {
      const t = (item.txt || item['文本'] || '').toLowerCase();
      const matchSearch = t.includes(searchTxt);
      const matchCat = selCat === '' || item.cat === selCat;
      const matchGen = selGen === '' || item.genre === selGen;
      const matchEmo = selEmo === '' || item.emo_cat === selEmo;
      return matchSearch && matchCat && matchGen && matchEmo;
    });
  }
  function renderTablePage() {
    const dataArray = filteredView.slice(0, RENDER_LIMIT);
    const total = dataArray.length;
    const pages = Math.max(1, Math.ceil(total / PAGE_SIZE));
    if (currentPage > pages) currentPage = pages;
    const start = (currentPage - 1) * PAGE_SIZE;
    const slice = dataArray.slice(start, start + PAGE_SIZE);
    const tbody = document.getElementById('traceback-body');
    let html = '';
    slice.forEach(item => {
      const apiPayload = itemApiPayload(item);
      const dataItem = encodeURIComponent(JSON.stringify(apiPayload));
      const uid = item._uid;
      html += `<tr class="data-row" data-uid="${uid}" data-item="${dataItem}">
        <td>${escapeHtml(item.poem_id)}</td>
        <td style="color:#2980b9;font-weight:bold;">${escapeHtml(item.txt || item['文本'])}</td>
        <td><b>${escapeHtml(item.title)}</b></td>
        <td><span class="tag">${escapeHtml(item.genre)}</span></td>
        <td>${escapeHtml(item.cat)}</td>
        <td>${escapeHtml(item.emotion)}</td>
        <td class="ops-cell">
          <button type="button" class="btn-link" onclick="openDetailModal('${uid}', event)">详情</button>
          <button type="button" class="btn-link" onclick="openAnalyzeDrawer('${uid}', event)">解析</button>
          <a href="javascript:void(0)" class="danger" onclick="event.stopPropagation();deleteItem('${uid}')">删除</a>
        </td>
      </tr>`;
    });
    if (total > RENDER_LIMIT) {
      html += `<tr><td colspan="7" style="text-align:center;color:#e74c3c;background:#fff3f3;">仅渲染前 ${RENDER_LIMIT} 条筛选结果，请缩小筛选
    }
    tbody.innerHTML = html || '<tr><td colspan="7" style="text-align:center;color:#999;">暂无数据</td></tr>';
```

```
    document.getElementById('record-count').innerText = `(共命中 ${total} 条记录 • 第 ${currentPage}/${pages} 页)`;
    document.getElementById('page-info').innerText = `每页 ${PAGE_SIZE} 条`;
    document.getElementById('page-prev').disabled = currentPage <= 1;
    document.getElementById('page-next').disabled = currentPage >= pages;
  }
  function openDetailModal(uid, ev) {
    if (ev) ev.stopPropagation();
    const item = ALL_DATA.find(i => i._uid === uid);
    if (!item) return;
    const w = item['文本'] || item.txt || '';
    document.getElementById('md-content').innerHTML = `
      <h2 style="color:var(--primary);margin-top:0;border-bottom:2px solid #eee;padding-bottom:12px;">《${escapeHtml(item.title)}》 — 意象详
      <div style="margin-bottom:10px;"><b style="color:#7f8c8d;width:120px;display:inline-block;">意象文本</b><span style="color:#e74c3c;font-
      <div style="margin-bottom:10px;"><b style="color:#7f8c8d;width:120px;display:inline-block;">大类归属</b><span>${escapeHtml(item.cat)}</s
      <div style="margin-bottom:10px;"><b style="color:#7f8c8d;width:120px;display:inline-block;">结构化标注</b>
        <div style="margin-top:8px;font-size:13px;line-height:1.7;color:#344;">
          词性 ${escapeHtml(String(item['词性'] ?? ''))} • 成分 ${escapeHtml(String(item['成分类型'] ?? ''))}<br>
          感知 ${escapeHtml(String(item['感知通道'] ?? ''))} • 素材 ${escapeHtml(String(item['素材类型'] ?? ''))}<br>
          内部结构 ${escapeHtml(String(item['内部结构'] ?? ''))} • 指涉 ${escapeHtml(String(item['指涉来源'] ?? ''))}<br>
          表现功能 ${escapeHtml(String(item['表现功能'] ?? ''))} • 结构组 ${escapeHtml(String(item['结构功能组'] ?? ''))}<br>
          文化流通 ${escapeHtml(String(item['文化流通性'] ?? ''))} • 跨文化性 ${escapeHtml(String(item['跨文化性'] ?? ''))}<br>
          认知强度 ${escapeHtml(String(item['认知强度'] ?? ''))} • 核心意象 ${escapeHtml(String(item['核心意象'] ?? ''))}<br>
          情感极性 ${escapeHtml(String(item['情感极性'] ?? ''))} • 类别 ${escapeHtml(String(item['情感类别'] ?? ''))} • 置信度 ${escapeHtml
          大类编码 ${escapeHtml(String(item['大类编码'] ?? ''))} • 子类 ${escapeHtml(String(item['子类编码'] ?? ''))}
        </div>
      </div>
      <div style="margin-bottom:10px;"><b style="color:#7f8c8d;width:120px;display:inline-block;">四层摘要</b><span style="color:#16a085;">${e
      <div style="margin-bottom:10px;"><b style="color:#7f8c8d;width:120px;display:inline-block;">情感倾向</b><span style="color:#e67e22;">${e
      <div><b style="color:#7f8c8d;display:block;margin-bottom:8px;">完整诗句</b>
        <div style="background:#f8f9fa;padding:14px;border-radius:8px;font-family:楷体,KaiTi,serif;font-size:17px;line-height:1.65;">${escapeH
      </div>`;
    document.getElementById('detailModal').style.display = 'block';
  }
  function mdToHtml(text) {
    if (!text) return '';
    if (typeof marked !== 'undefined' && typeof marked.parse === 'function') {
      try {
        return marked.parse(text, { breaks: true });
      } catch (e) {
        return '<p>' + escapeHtml(text) + '</p>';
      }
    }
    return '<p>' + escapeHtml(text) + '</p>';
  }
  function getItemPayloadByUid(uid) {
    const tr = document.querySelector(`tr.data-row[data-uid="${uid}"]`);
    try {
      const raw = tr && tr.getAttribute('data-item');
      if (raw) return JSON.parse(decodeURIComponent(raw));
    } catch (e) { console.warn(e); }
    return itemApiPayload(ALL_DATA.find(i => i._uid === uid));
  }
  function buildTagChips(item) {
    const chips = [];
    const pc = item['感知通道'];
    if (pc) chips.push(String(pc));
    const cat = item.cat || '';
    const shortCat = cat.includes('-') ? cat.split('-').pop().trim() : cat;
    if (shortCat) chips.push(shortCat);
    const fx = item['表现功能'];
    if (fx) chips.push(String(fx));
    const cs = item['认知强度'];
    if (cs !== '' && cs != null) chips.push('认知强度' + cs);
    const emo = item['情感类别'] || item.emo_cat || '';
```

```
    const pol = item['情感极性'];
    let emoChip = '';
    if (emo && pol !== '' && pol != null) emoChip = String(emo) + String(pol);
    else if (emo) emoChip = String(emo);
    else if (pol !== '' && pol != null) emoChip = String(pol);
    if (emoChip) chips.push(emoChip);
    return chips.filter(Boolean);
  }
  function fillDrawerChrome(item) {
    const w = item['文本'] || item.txt || '—';
    document.getElementById('drawer-img-text').textContent = w;
    document.getElementById('drawer-poem-meta').textContent = `《${item.title || ''}》 ${item.genre || ''}`.trim();
    document.getElementById('drawer-line-text').textContent = '诗句：' + (item.line || '—');
    const tagBox = document.getElementById('drawer-tags');
    const chips = buildTagChips(item);
    tagBox.innerHTML = chips.map(c => `<span class="tag-chip">${escapeHtml(c)}</span>`).join('');
  }
  function scrollDrawerToBottom() {
    const el = document.getElementById('drawer-chat-scroll');
    if (el) el.scrollTop = el.scrollHeight;
  }
  function renderDrawerMessagesFromHistory(hist) {
    const box = document.getElementById('drawer-messages');
    box.innerHTML = '';
    if (!hist || !hist.length) return;
    hist.forEach((m) => {
      if (m.role === 'user') {
        const row = document.createElement('div');
        row.className = 'msg-row user';
        row.innerHTML = `<div class="msg-bubble user">${escapeHtml(m.content)}</div>`;
        box.appendChild(row);
      } else if (m.role === 'assistant') {
        const row = document.createElement('div');
        row.className = 'msg-row assistant';
        const bubble = document.createElement('div');
        bubble.className = 'msg-bubble assistant drawer-md';
        bubble.innerHTML = mdToHtml(m.content);
        row.appendChild(bubble);
        box.appendChild(row);
      }
    });
    scrollDrawerToBottom();
  }
  function openDrawerDom() {
    document.getElementById('drawer-overlay').classList.add('show');
    const dr = document.getElementById('analyze-drawer');
    dr.classList.add('open');
    dr.setAttribute('aria-hidden', 'false');
  }
  function closeAnalyzeDrawer() {
    document.getElementById('drawer-overlay').classList.remove('show');
    const dr = document.getElementById('analyze-drawer');
    dr.classList.remove('open');
    dr.setAttribute('aria-hidden', 'true');
    currentCacheKey = null; // 仅仅是视角切走，后台不中断
  }
  function restoreFromCache(cacheKey) {
    const e = ANALYZE_CACHE[cacheKey];
    if (!e) return;
    const box = document.getElementById('drawer-messages');
    if (e.rendered) {
      box.innerHTML = e.rendered;
    } else if (e.messages && e.messages.length) {
      renderDrawerMessagesFromHistory(e.messages);
```

```
      e.rendered = box.innerHTML;
    } else {
      box.innerHTML = '';
    }
    scrollDrawerToBottom();
  }
  function openAnalyzeDrawer(uid, ev) {
    if (ev) ev.stopPropagation();
    const item = getItemPayloadByUid(uid);
    const cacheKey = makeCacheKey(item);
    currentCacheKey = cacheKey;
    fillDrawerChrome(item);
    openDrawerDom();
    let ent = ANALYZE_CACHE[cacheKey];
    if (!ent) {
      ent = {
        item: JSON.parse(JSON.stringify(item)),
        messages: [],
        rendered: '',
        done: false,
        isStreaming: false,
        abortController: null
      };
      ANALYZE_CACHE[cacheKey] = ent;
    }
    if (ent.isStreaming) {
      restoreFromCache(cacheKey);
      return;
    }
    if (isCacheAnalysisComplete(ent)) {
      if (ent.done !== true) ent.done = true;
      restoreFromCache(cacheKey);
      return;
    }
    ent.messages = [];
    ent.rendered = '';
    ent.done = false;
    document.getElementById('drawer-messages').innerHTML = '';
    document.getElementById('drawer-followup').value = '';
    runDrawerInitialStream(cacheKey, item, ent);
  }
  async function runDrawerInitialStream(cacheKey, item, ent) {
    ent.isStreaming = true;
    ent.abortController = new AbortController();
    const bubbleId = 'bubble-' + Math.random().toString(36).substr(2, 9);
    let acc = '';
    const userHtml = `<div class="msg-row user"><div class="msg-bubble user">${escapeHtml(DEFAULT_ANALYZE_PROMPT)}</div></div>`;
    const asstHtml = `<div class="msg-row assistant"><div class="msg-bubble assistant drawer-md" id="${bubbleId}">...</div></div>`;
    ent.rendered = userHtml + asstHtml;
    if (currentCacheKey === cacheKey) {
      document.getElementById('drawer-messages').innerHTML = ent.rendered;
    }
    try {
      const res = await fetch('/api/analyze', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ item, history: [], followup: DEFAULT_ANALYZE_PROMPT }),
        signal: ent.abortController.signal
      });
      await readSSE(res, (obj) => {
        if (obj.type === 'chunk' && obj.text) {
          acc += obj.text;
        } else if (obj.type === 'error') {
          acc += '\n\n【错误】' + (obj.message || '');
```

```
        }
        ent.rendered = userHtml + `<div class="msg-row assistant"><div class="msg-bubble assistant drawer-md" id="${bubbleId}">${mdToHtml(acc)
        if (currentCacheKey === cacheKey) {
          const bubble = document.getElementById(bubbleId);
          if (bubble) bubble.innerHTML = mdToHtml(acc);
          scrollDrawerToBottom();
        }
      }, { signal: ent.abortController.signal });
      if (!ent.abortController.signal.aborted) {
        ent.messages.push(
          { role: 'user', content: DEFAULT_ANALYZE_PROMPT },
          { role: 'assistant', content: acc }
        );
        ent.done = true;
      }
    } catch (e) {
      if (e.name !== 'AbortError') {
        acc += '\n\n<p class="danger">请求失败：' + escapeHtml(String(e)) + '</p>';
        ent.rendered = userHtml + `<div class="msg-row assistant"><div class="msg-bubble assistant drawer-md" id="${bubbleId}">${mdToHtml(acc)
        if (currentCacheKey === cacheKey) {
          const bubble = document.getElementById(bubbleId);
          if (bubble) bubble.innerHTML = mdToHtml(acc);
        }
      }
    } finally {
      ent.isStreaming = false;
      ent.abortController = null;
      if (currentCacheKey === cacheKey) scrollDrawerToBottom();
    }
  }
  async function sendDrawerFollowup() {
    const cacheKey = currentCacheKey;
    if (!cacheKey) return;
    const ent = ANALYZE_CACHE[cacheKey];
    if (!ent || !ent.messages.length || ent.isStreaming) return;
    const inp = document.getElementById('drawer-followup');
    const q = (inp.value || '').trim();
    if (!q) return;
    inp.value = '';
    const hist = JSON.parse(JSON.stringify(ent.messages));
    const item = ent.item || {};
    ent.isStreaming = true;
    ent.abortController = new AbortController();
    const bubbleId = 'bubble-' + Math.random().toString(36).substr(2, 9);
    let acc = '';
    const newQ = `<div class="msg-row user"><div class="msg-bubble user">${escapeHtml(q)}</div></div>`;
    const newA = `<div class="msg-row assistant"><div class="msg-bubble assistant drawer-md" id="${bubbleId}">...</div></div>`;
    const baseRendered = ent.rendered;
    ent.rendered = baseRendered + newQ + newA;
    if (currentCacheKey === cacheKey) {
       const box = document.getElementById('drawer-messages');
       box.insertAdjacentHTML('beforeend', newQ + newA);
       scrollDrawerToBottom();
    }
    try {
      const res = await fetch('/api/analyze', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ item, history: hist, followup: q }),
        signal: ent.abortController.signal
      });
      await readSSE(res, (obj) => {
        if (obj.type === 'chunk' && obj.text) {
          acc += obj.text;
```

```
          } else if (obj.type === 'error') {
            acc += '\n\n【错误】' + (obj.message || '');
          }
          ent.rendered = baseRendered + newQ + `<div class="msg-row assistant"><div class="msg-bubble assistant drawer-md" id="${bubbleId}">${mo
          if (currentCacheKey === cacheKey) {
            const bubble = document.getElementById(bubbleId);
            if (bubble) bubble.innerHTML = mdToHtml(acc);
            scrollDrawerToBottom();
          }
        }, { signal: ent.abortController.signal });
        if (!ent.abortController.signal.aborted) {
          ent.messages.push({ role: 'user', content: q }, { role: 'assistant', content: acc });
        }
      } catch (e) {
        if (e.name !== 'AbortError') {
           acc += '\n\n<p class="danger">请求失败：' + escapeHtml(String(e)) + '</p>';
           ent.rendered = baseRendered + newQ + `<div class="msg-row assistant"><div class="msg-bubble assistant drawer-md" id="${bubbleId}">${m
           if (currentCacheKey === cacheKey) {
             const bubble = document.getElementById(bubbleId);
             if(bubble) bubble.innerHTML = mdToHtml(acc);
           }
        }
      } finally {
        ent.isStreaming = false;
        ent.abortController = null;
        if (currentCacheKey === cacheKey) scrollDrawerToBottom();
      }
    }
    function escapeHtml(s) {
      if (!s) return '';
      return String(s).replace(/&/g,'&amp;').replace(/</g,'&lt;').replace(/>/g,'&gt;');
    }
    function changePage(delta) {
      const dataArray = filteredView.slice(0, RENDER_LIMIT);
      const pages = Math.max(1, Math.ceil(dataArray.length / PAGE_SIZE));
      currentPage = Math.min(pages, Math.max(1, currentPage + delta));
      renderTablePage();
    }
    function applyFilters() {
      filteredView = getFiltered();
      currentPage = 1;
      const searchTxt = document.getElementById('searchTxt').value;
      const selCat = document.getElementById('filterCat').value;
      const selGen = document.getElementById('filterGenre').value;
      const selEmo = document.getElementById('filterEmotion').value;
      const isFiltered = !!(searchTxt || selCat || selGen || selEmo);
      document.getElementById('resetBtn').style.display = isFiltered ? 'inline-flex' : 'none';
      renderTablePage();
    }
    function updateCharts() {
      let individualCounts = {};
      let categoryCounts = {};
      ALL_DATA.forEach(item => {
        const tk = item.txt || item['文本'] || '';
        individualCounts[tk] = (individualCounts[tk] || 0) + 1;
        categoryCounts[item.cat] = (categoryCounts[item.cat] || 0) + 1;
      });
      const sortedInd = Object.entries(individualCounts).sort((a,b) => b[1]-a[1]).slice(0,50);
      const sortedCat = Object.entries(categoryCounts).sort((a,b) => b[1]-a[1]);
      indChart.setOption({
        tooltip: { trigger:'axis', axisPointer:{ type:'shadow' } },
        grid: { top:'10%', left:'5%', right:'5%', bottom:'25%' },
        xAxis: { type:'category', data: sortedInd.map(x=>x[0]), axisLabel:{ interval:0, rotate:30 } },
        yAxis: { type:'value' },
```

```
      dataZoom: [{ type:'slider', show:true, xAxisIndex:[0], start:0, end:40, bottom:0 }],
      series: [{ name:'频次', type:'bar', barWidth:'60%', data: sortedInd.map(x=>x[1]), itemStyle:{ color:'#3498db' } }]
    }, true);
    catChart.setOption({
      tooltip: { trigger:'axis', axisPointer:{ type:'shadow' } },
      grid: { top:'10%', left:'5%', right:'5%', bottom:'15%' },
      xAxis: { type:'category', data: sortedCat.map(x=>x[0]), axisLabel:{ interval:0, rotate:15 } },
      yAxis: { type:'value' },
      series: [{ name:'频次', type:'bar', barWidth:'50%', data: sortedCat.map(x=>x[1]), itemStyle:{ color:'#9b59b6' } }]
    }, true);
  }
  function refreshAll() {
    updateUIStats();
    populateSelects();
    filteredView = getFiltered();
    renderTablePage();
    updateCharts();
  }
  function deleteItem(uid) {
    const targetIndex = ALL_DATA.findIndex(item => item._uid === uid);
    if (targetIndex === -1) return;
    const targetItem = ALL_DATA.splice(targetIndex, 1)[0];
    DELETED_DATA.push(targetItem);
    const ck = makeCacheKey(targetItem);
    if (ck && ANALYZE_CACHE[ck]) {
      const ent = ANALYZE_CACHE[ck];
      if (ent.isStreaming && ent.abortController) {
        ent.abortController.abort();
      }
      delete ANALYZE_CACHE[ck];
    }
    if (currentCacheKey === ck) {
      closeAnalyzeDrawer();
    }
    refreshAll();
  }
  function restoreItem(uid) {
    const targetIndex = DELETED_DATA.findIndex(item => item._uid === uid);
    if (targetIndex === -1) return;
    const targetItem = DELETED_DATA.splice(targetIndex, 1)[0];
    ALL_DATA.push(targetItem);
    refreshAll();
    renderRecycleBin();
  }
  function renderRecycleBin() {
    const tbody = document.getElementById('recycle-body');
    let html = '';
    if (DELETED_DATA.length === 0) {
      html = '<tr><td colspan="5" style="text-align:center;padding:24px;color:#999;">回收站为空</td></tr>';
    } else {
      DELETED_DATA.forEach(item => {
        html += `<tr>
          <td>${escapeHtml(item.poem_id)}</td>
          <td style="color:#e74c3c;font-weight:bold;text-decoration:line-through;">${escapeHtml(item.txt)}</td>
          <td>${escapeHtml(item.title)}</td>
          <td>${escapeHtml(item.cat)}</td>
          <td><button type="button" class="btn-action" style="background:var(--success);color:#fff;padding:6px 10px;font-size:12px;" onclick="
        </tr>`;
      });
    }
    tbody.innerHTML = html;
  }
  function openRecycle() {
    renderRecycleBin();
```

```
    document.getElementById('recycleModal').style.display = 'block';
  }
  function closeModal(id) {
    document.getElementById(id).style.display = 'none';
  }
  function resetFilters() {
    document.getElementById('searchTxt').value = '';
    document.getElementById('filterCat').value = '';
    document.getElementById('filterGenre').value = '';
    document.getElementById('filterEmotion').value = '';
    applyFilters();
  }
  function filterFromChart(text, isCategory) {
    resetFilters();
    if (isCategory) document.getElementById('filterCat').value = text;
    else document.getElementById('searchTxt').value = text;
    applyFilters();
    document.getElementById('dataTable').scrollIntoView({ behavior:'smooth', block:'start' });
  }
  function toggleFullScreen(elemId) {
    const elem = document.getElementById(elemId);
    if (!document.fullscreenElement) {
      elem.requestFullscreen().catch(err => alert('全屏请求失败: ' + err.message));
    } else {
      document.exitFullscreen();
    }
  }
  document.addEventListener('fullscreenchange', () => {
    const isFull = !!document.fullscreenElement;
    document.querySelectorAll('.chart-container').forEach(el => {
      el.style.height = isFull ? 'calc(100vh - 80px)' : '400px';
    });
    if (indChart) indChart.resize();
    if (catChart) catChart.resize();
  });
  async function readSSE(response, onJson, opts) {
    const signal = opts && opts.signal;
    if (signal && signal.aborted) return;
    const reader = response.body.getReader();
    const dec = new TextDecoder();
    let buf = '';
    while (true) {
      if (signal && signal.aborted) {
        try { await reader.cancel(); } catch (e) { /* ignore */ }
        break;
      }
      let readResult;
      try {
        readResult = await reader.read();
      } catch (err) {
        if (signal && signal.aborted) break;
        throw err;
      }
      const { value, done } = readResult;
      if (done) break;
      buf += dec.decode(value, { stream: true });
      const parts = buf.split('\n\n');
      buf = parts.pop() || '';
      for (const block of parts) {
        const lines = block.split('\n');
        for (const line of lines) {
          if (line.startsWith('data: ')) {
            try {
              const obj = JSON.parse(line.slice(6));
```

```
                onJson(obj);
              } catch (e) { console.warn(e); }
            }
          }
        }
      }
    }
    async function runRagFirst() {
      const q = (document.getElementById('rag-question').value || '').trim();
      const hint = document.getElementById('rag-hint');
      hint.style.display = 'none';
      if (!q || ragBusy) return;
      ragBusy = true;
      ragConv = [];
      document.getElementById('rag-answer').innerText = '';
      document.getElementById('rag-poems').innerHTML = '';
      document.getElementById('ai-split').classList.add('active');
      let ans = '';
      try {
        const res = await fetch('/api/ask', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ question: q, history: [] })
        });
        await readSSE(res, (obj) => {
          if (obj.type === 'poems') {
            ragPoems = obj.poems || [];
            renderRagPoems();
          } else if (obj.type === 'chunk' && obj.text) {
            ans += obj.text;
            document.getElementById('rag-answer').innerText = ans;
            const el = document.getElementById('rag-answer');
            el.scrollTop = el.scrollHeight;
          } else if (obj.type === 'error') {
            hint.innerText = obj.message || '出错';
            hint.style.display = 'block';
          }
        });
        if (ans) ragConv = [
          { role: 'user', content: q },
          { role: 'assistant', content: ans }
        ];
      } catch (e) {
        hint.innerText = '请求失败：' + e;
        hint.style.display = 'block';
      } finally {
        ragBusy = false;
      }
    }
    function renderRagPoems() {
      const box = document.getElementById('rag-poems');
      if (!ragPoems.length) {
        box.innerHTML = '<p style="color:#999;">暂无检索结果</p>';
        return;
      }
      box.innerHTML = ragPoems.map((p) => `
        <div class="poem-card" onclick="this.classList.toggle('expanded')">
          <h4>《${escapeHtml(p['标题'])}》 ${escapeHtml(p['作者'])}</h4>
          <div class="sub">相似度：${p['相似度']}</div>
          <div class="body">${escapeHtml((p['原文'] || '').slice(0, 120))}${(p['原文']||'').length>120?'…':''}</div>
          <div class="poem-detail">${escapeHtml(p['原文'] || '')}</div>
          <div class="sim">点击卡片展开全文</div>
        </div>
      `).join('');
```

```
}
async function runRagFollowup() {
  const q = (document.getElementById('rag-followup').value || '').trim();
  const firstQ = document.getElementById('rag-question').value.trim();
  if (!q || ragBusy || !firstQ || !ragConv.length) return;
  ragBusy = true;
  document.getElementById('rag-followup').value = '';
  const ansEl = document.getElementById('rag-answer');
  ansEl.innerText += `\n\n【问】${q}\n\n【答】`;
  let acc = '';
  const markPos = ansEl.innerText.length;
  const hist = ragConv.map(m => ({ role: m.role, content: m.content }));
  try {
    const res = await fetch('/api/ask', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ question: q, history: hist })
    });
    await readSSE(res, (obj) => {
      if (obj.type === 'chunk' && obj.text) {
        acc += obj.text;
        ansEl.innerText = ansEl.innerText.slice(0, markPos) + acc;
        ansEl.scrollTop = ansEl.scrollHeight;
      } else if (obj.type === 'error') {
        ansEl.innerText += '\n【错误】' + (obj.message || '');
      }
    });
    ragConv.push({ role: 'user', content: q });
    ragConv.push({ role: 'assistant', content: acc });
  } catch (e) {
    ansEl.innerText += '\n请求失败：' + e;
  } finally {
    ragBusy = false;
  }
}
/* Tab switch */
document.querySelectorAll('.tab-btn').forEach(btn => {
  btn.addEventListener('click', () => {
    document.querySelectorAll('.tab-btn').forEach(b => b.classList.remove('active'));
    btn.classList.add('active');
    const tab = btn.getAttribute('data-tab');
    document.querySelectorAll('.tab-pane').forEach(p => {
      p.classList.remove('visible');
    });
    if (tab === 'graph') {
      document.getElementById('tab-graph').classList.add('visible');
      setTimeout(() => { if (indChart) indChart.resize(); if (catChart) catChart.resize(); }, 200);
    } else {
      document.getElementById('tab-ai').classList.add('visible');
    }
  });
});
window.addEventListener('resize', () => {
  if (indChart) indChart.resize();
  if (catChart) catChart.resize();
});
loadApiData().then(() => {
  if (indChart) indChart.on('click', p => filterFromChart(p.name, false));
  if (catChart) catChart.on('click', p => filterFromChart(p.name, true));
});
/* --- 抽屉宽度拖拽 (IDE Style) --- */
const drw = document.getElementById('analyze-drawer');
const resizer = document.getElementById('drawer-resizer');
let isResizing = false;
```

```
    resizer.addEventListener('mousedown', (e) => {
      isResizing = true;
      resizer.classList.add('active');
      drw.classList.add('no-transition');
      document.body.style.cursor = 'ew-resize';
      document.body.style.userSelect = 'none';
    });
    document.addEventListener('mousemove', (e) => {
      if (!isResizing) return;
      let newWidth = window.innerWidth - e.clientX;
      if (newWidth < 320) newWidth = 320;
      if (newWidth > window.innerWidth - 80) newWidth = window.innerWidth - 80;
      drw.style.width = newWidth + 'px';
    });
    document.addEventListener('mouseup', () => {
      if (isResizing) {
        isResizing = false;
        resizer.classList.remove('active');
        drw.classList.remove('no-transition');
        document.body.style.cursor = '';
        document.body.style.userSelect = '';
      }
    });
  </script>
</body>
</html>
```

-- 文件结束：templates/index.html --

文件：templates/admin.html (363 行, 12.3 KB)

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>系统管理 — 唐诗意象智能分析系统</title>
  <script src="https://cdn.jsdelivr.net/npm/echarts@5.5.0/dist/echarts.min.js"></script>
  <style>
    :root {
      --primary: #2c3e50;
      --accent: #3498db;
      --bg: #f4f7f6;
      --card: #ffffff;
      --success: #2ecc71;
      --warn: #e67e22;
      --danger: #e74c3c;
    }
    * { box-sizing: border-box; }
    body {
      font-family: 'Helvetica Neue', Arial, sans-serif;
      background: var(--bg);
      margin: 0;
      padding: 0;
      color: #333;
    }
    .navbar {
      background: var(--primary);
      color: #fff;
      padding: 16px 28px;
      display: flex;
      align-items: center;
      justify-content: space-between;
    }
    .navbar h1 { margin: 0; font-size: 20px; }
    .navbar a { color: #bdc3c7; text-decoration: none; font-size: 14px; }
    .navbar a:hover { color: #fff; }
```

```
    .container { max-width: 1400px; margin: 24px auto; padding: 0 20px; }
    .stats-grid {
      display: grid;
      grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
      gap: 16px;
      margin-bottom: 24px;
    }
    .stat-card {
      background: var(--card);
      border-radius: 10px;
      padding: 20px;
      box-shadow: 0 2px 8px rgba(0,0,0,0.06);
      text-align: center;
    }
    .stat-card .num { font-size: 32px; font-weight: 800; color: var(--accent); }
    .stat-card .label { font-size: 13px; color: #888; margin-top: 6px; }
    .card {
      background: var(--card);
      border-radius: 10px;
      padding: 24px;
      margin-bottom: 24px;
      box-shadow: 0 2px 8px rgba(0,0,0,0.06);
    }
    .card-title {
      margin: 0 0 16px;
      font-size: 18px;
      color: var(--primary);
      border-left: 4px solid var(--accent);
      padding-left: 12px;
    }
    .grid-2 {
      display: grid;
      grid-template-columns: 1fr 1fr;
      gap: 24px;
    }
    @media (max-width: 800px) { .grid-2 { grid-template-columns: 1fr; } }
    .chart-box { height: 340px; width: 100%; }
    .prop-table { width: 100%; border-collapse: collapse; font-size: 14px; }
    .prop-table th, .prop-table td {
      border: 1px solid #eee;
      padding: 10px 14px;
      text-align: left;
    }
    .prop-table th { background: #fafafa; font-weight: 600; }
    .prop-table tr:hover { background: #f8f9fa; }
    .btn {
      padding: 10px 22px;
      border-radius: 8px;
      border: none;
      font-weight: 600;
      cursor: pointer;
      transition: 0.2s;
      font-size: 14px;
    }
    .btn-primary { background: var(--accent); color: #fff; }
    .btn-primary:hover { filter: brightness(1.05); }
    .btn-success { background: var(--success); color: #fff; }
    .btn-success:hover { filter: brightness(1.05); }
    .btn-warn { background: var(--warn); color: #fff; }
    .btn-warn:hover { filter: brightness(1.05); }
    .btn-danger { background: var(--danger); color: #fff; }
    .btn-danger:hover { filter: brightness(1.05); }
    .btn-sm { padding: 6px 14px; font-size: 12px; }
    .action-bar {
```

```
      display: flex;
      gap: 12px;
      flex-wrap: wrap;
      margin-bottom: 16px;
    }
    .tag {
      background: #ecf0f1;
      padding: 3px 10px;
      border-radius: 20px;
      font-size: 12px;
      display: inline-block;
    }
    .tag-green { background: #d5f5e3; color: #27ae60; }
    .tag-red { background: #fadbd8; color: #c0392b; }
    .status-dot {
      display: inline-block;
      width: 10px; height: 10px;
      border-radius: 50%;
      margin-right: 6px;
    }
    .status-dot.online { background: var(--success); }
    .export-list { max-height: 300px; overflow-y: auto; }
    .toast {
      position: fixed;
      bottom: 30px;
      right: 30px;
      background: var(--primary);
      color: #fff;
      padding: 14px 24px;
      border-radius: 10px;
      font-size: 14px;
      opacity: 0;
      transform: translateY(20px);
      transition: 0.3s;
      z-index: 999;
    }
    .toast.show { opacity: 1; transform: translateY(0); }
  </style>
</head>
<body>
  <nav class="navbar">
    <h1> 系统管理后台</h1>
    <div>
      <a href="/">← 返回前台</a>
    </div>
  </nav>
  <div class="container">
    <div id="stats-panel" class="stats-grid"></div>
    <div class="grid-2">
      <!-- 分类分布 -->
      <div class="card">
        <h2 class="card-title">意象分类域分布</h2>
        <div id="chart-cat" class="chart-box"></div>
      </div>
      <!-- 情感分布 -->
      <div class="card">
        <h2 class="card-title">情感类别分布</h2>
        <div id="chart-emo" class="chart-box"></div>
      </div>
    </div>
    <!-- 系统配置 -->
    <div class="card">
      <h2 class="card-title">系统配置</h2>
      <table class="prop-table" id="config-table"></table>
```

```
    </div>
    <!-- 操作区 -->
    <div class="card">
      <h2 class="card-title">操作面板</h2>
      <div class="action-bar">
        <button class="btn btn-primary" onclick="refreshData()"> 刷新数据缓存</button>
        <button class="btn btn-success" onclick="exportData('csv')"> 导出 CSV</button>
        <button class="btn btn-success" onclick="exportData('json')"> 导出 JSON</button>
        <button class="btn btn-success" onclick="exportData('report')"> 导出分析报告</button>
        <button class="btn btn-warn" onclick="clearCache()"> 清理缓存</button>
      </div>
    </div>
    <!-- 导出文件列表 -->
    <div class="card">
      <h2 class="card-title">导出文件列表</h2>
      <div class="action-bar">
        <button class="btn btn-danger btn-sm" onclick="clearExports()">清空全部</button>
      </div>
      <div class="export-list" id="export-list"><p style="color:#999;">加载中...</p></div>
    </div>
  </div>
  <div id="toast" class="toast"></div>
  <script>
    let catChart = null;
    let emoChart = null;
    async function fetchJSON(url) {
      const res = await fetch(url);
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      return res.json();
    }
    function showToast(msg, isError) {
      const el = document.getElementById('toast');
      el.textContent = msg;
      el.style.background = isError ? '#e74c3c' : '#2c3e50';
      el.classList.add('show');
      setTimeout(() => el.classList.remove('show'), 3000);
    }
    async function loadStatus() {
      try {
        const s = await fetchJSON('/admin/api/status');
        const panel = document.getElementById('stats-panel');
        panel.innerHTML = `
          <div class="stat-card"><div class="num">${s.data.total_poems}</div><div class="label">净解析诗歌</div></div>
          <div class="stat-card"><div class="num">${s.data.total_images}</div><div class="label">提取意象条目</div></div>
          <div class="stat-card"><div class="num">${s.data.category_count}</div><div class="label">分类维度</div></div>
          <div class="stat-card"><div class="num">${s.data.known_authors_count}</div><div class="label">已知诗人</div></div>
          <div class="stat-card">
            <div class="num">${s.rag_database.exists ? '<span style="color:#2ecc71;">V</span>' : '<span style="color:#e74c3c;">X</span>'}</div>
            <div class="label">向量数据库 (${s.rag_database.file_count || 0} 文件)</div>
          </div>
          <div class="stat-card">
            <div class="num">${s.cache.memory_entries}</div>
            <div class="label">内存缓存条目</div>
          </div>
          <div class="stat-card">
            <div class="num">${s.exports.file_count}</div>
            <div class="label">导出文件</div>
          </div>
          <div class="stat-card">
            <div class="num">${s.poem_json.file_count}</div>
            <div class="label">诗歌数据文件</div>
          </div>
        `;
        const configTable = document.getElementById('config-table');
```

```
      configTable.innerHTML = Object.entries(s.config).map(([k, v]) =>
        `<tr><td>${k}</td><td>${v}</td></tr>`
      ).join('');
    } catch (e) {
      showToast('加载状态失败: ' + e.message, true);
    }
  }
  async function loadStats() {
    try {
      const stats = await fetchJSON('/admin/api/stats');
      const catData = (stats.category_stats || []).slice(0, 30);
      const emoData = (stats.emotion_stats || []).slice(0, 20);
      if (!catChart) {
        catChart = echarts.init(document.getElementById('chart-cat'));
      }
      catChart.setOption({
        tooltip: { trigger: 'axis', axisPointer: { type: 'shadow' } },
        grid: { top: '8%', left: '15%', right: '5%', bottom: '20%' },
        xAxis: { type: 'value' },
        yAxis: { type: 'category', data: catData.map(d => d.category).reverse(), axisLabel: { fontSize: 11 } },
        series: [{ type: 'bar', data: catData.map(d => d.count).reverse(), itemStyle: { color: '#9b59b6' }, barWidth: '60%' }]
      }, true);
      if (!emoChart) {
        emoChart = echarts.init(document.getElementById('chart-emo'));
      }
      emoChart.setOption({
        tooltip: { trigger: 'axis', axisPointer: { type: 'shadow' } },
        grid: { top: '8%', left: '5%', right: '5%', bottom: '20%' },
        xAxis: { type: 'category', data: emoData.map(d => d.emotion), axisLabel: { interval: 0, rotate: 20, fontSize: 11 } },
        yAxis: { type: 'value' },
        series: [{ type: 'bar', data: emoData.map(d => d.count), itemStyle: { color: '#e67e22' }, barWidth: '50%' }]
      }, true);
      window.addEventListener('resize', () => { catChart && catChart.resize(); emoChart && emoChart.resize(); });
    } catch (e) {
      showToast('加载统计数据失败: ' + e.message, true);
    }
  }
  async function loadExports() {
    try {
      const data = await fetchJSON('/admin/api/exports');
      const list = document.getElementById('export-list');
      if (!data.exports || !data.exports.length) {
        list.innerHTML = '<p style="color:#999;">暂无导出文件</p>';
        return;
      }
      list.innerHTML = '<table class="prop-table">' +
        '<thead><tr><th>文件名</th><th>大小</th><th>修改时间</th></tr></thead><tbody>' +
        data.exports.map(f => {
          const size = f.size > 1024 ? (f.size / 1024).toFixed(1) + ' KB' : f.size + ' B';
          const time = new Date(f.mtime * 1000).toLocaleString();
          return `<tr><td>${f.name}</td><td>${size}</td><td>${time}</td></tr>`;
        }).join('') + '</tbody></table>';
    } catch (e) {
      document.getElementById('export-list').innerHTML = '<p style="color:#e74c3c;">加载失败</p>';
    }
  }
  async function refreshData() {
    try {
      const res = await fetchJSON('/admin/api/refresh');
      showToast(`数据已刷新: ${res.total_poems} 首诗歌, ${res.total_images} 条意象`);
      loadStatus();
      loadStats();
    } catch (e) {
      showToast('刷新失败: ' + e.message, true);
```

```
      }
    }
    async function exportData(fmt) {
      try {
        const res = await fetch('/admin/api/exports', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ format: fmt })
        });
        const data = await res.json();
        if (data.error) { showToast(data.error, true); return; }
        showToast(`导出成功: ${data.file} (${(data.size / 1024).toFixed(1)} KB)`);
        loadExports();
      } catch (e) {
        showToast('导出失败: ' + e.message, true);
      }
    }
    async function clearCache() {
      try {
        const res = await fetch('/admin/api/cache/clear', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ type: 'all' })
        });
        const data = await res.json();
        showToast(data.message);
        loadStatus();
      } catch (e) {
        showToast('清理失败: ' + e.message, true);
      }
    }
    async function clearExports() {
      if (!confirm('确定要清空所有导出文件吗?')) return;
      try {
        const res = await fetch('/admin/api/exports/clear', { method: 'POST' });
        const data = await res.json();
        showToast(data.message);
        loadExports();
      } catch (e) {
        showToast('清空失败: ' + e.message, true);
      }
    }
    // 初始化
    loadStatus();
    loadStats();
    loadExports();
  </script>
</body>
</html>
-- 文件结束：templates/admin.html --
文件：json_to_3csv.py  (163 行, 7.0 KB)
import json
import csv
import os
import glob
def aggregate_jsons_to_csv():
    base_dir = os.path.dirname(os.path.abspath(__file__))
    json_folder = os.path.join(base_dir, 'json_data')
    if not os.path.exists(json_folder):
        print(f" 找不到文件夹: {json_folder}，请创建并放入JSON文件！")
        return
    json_files = glob.glob(os.path.join(json_folder, '*.json'))
    if not json_files:
        print(" 文件夹里没有找到任何 .json 文件！")
```

```
        return
    print(f" 发现 {len(json_files)} 个 JSON 文件，正在执行安全解析与聚合...")
    # --- 1. 定义三张表的表头 ---
    header_a = [
        "诗歌编号", "标题", "作者", "分类标签", "诗行编号",
        "单元编号", "文本", "行内位置", "词性", "成分类型",
        "是否意象", "大类编码", "子类编码", "感知通道", "素材类型",
        "内部结构", "指涉来源", "表现功能", "文化流通性", "跨文化性",
        "认知强度", "核心意象", "结构功能组", "情感极性", "情感类别", "情感置信度"
    ]
    header_b = [
        "诗歌编号", "标题", "作者", "分类标签",
        "诗行编号", "诗行原文",
        "平均情感极性", "主导情感", "情感波动值"
    ]
    header_c = [
        "诗歌编号", "标题", "作者", "关系编号",
        "来源单元编号", "来源文本",
        "目标单元编号", "目标文本",
        "关系类型", "关系强度"
    ]
    rows_a, rows_b, rows_c = [], [], []
    # 全局计数器（解决所有文件都从 P01 和 U001 开始的 Bug）
    global_poem_idx = 1
    global_unit_idx = 1
    global_rel_idx = 1
    global_traj_idx = 1
    # --- 2. 遍历所有 JSON 文件 ---
    for file_path in json_files:
        try:
            with open(file_path, 'r', encoding='utf-8') as f:
                data = json.load(f)
        except Exception as e:
            print(f" 解析失败，跳过损坏的 JSON 文件 {os.path.basename(file_path)}: {e}")
            continue
        poems = data.get("诗歌集", []) if isinstance(data, dict) else data
        # --- 3. 遍历文件内的诗歌 ---
        for p in poems:
            # 分配全局唯一的诗歌 ID
            new_p_id = f"P{global_poem_idx:03d}"
            global_poem_idx += 1
            title = p.get("标题", "")
            author = p.get("作者", "")
            tag = p.get("分类标签", "")
            base_meta = [new_p_id, title, author, tag]
            # 建立旧 ID 到新 ID 的映射字典，防止指针错乱
            line_id_map = {}
            unit_id_map = {}
            unit_text_map = {} # 用于关系表反查文本
            # 3.1 提取诗行并映射新 ID
            line_text_map = {}
            for i, line in enumerate(p.get("诗行", []), 1):
                old_l_id = line.get("诗行编号", "")
                new_l_id = f"{new_p_id}_L{i:02d}"
                line_id_map[old_l_id] = new_l_id
                line_text_map[new_l_id] = line.get("原文", "")
            # 3.2 解析表 A：分析单元 (并映射全局单元 ID)
            for u in p.get("分析单元", []):
                old_u_id = u.get("单元编号", "")
                new_u_id = f"U{global_unit_idx:05d}"
                global_unit_idx += 1
                unit_id_map[old_u_id] = new_u_id
                text_val = u.get("文本", "")
                unit_text_map[new_u_id] = text_val
```

```
                # 转换对应的诗行 ID
                old_u_line = u.get("诗行编号", "")
                mapped_l_id = line_id_map.get(old_u_line, old_u_line)
                rows_a.append([
                    new_p_id, title, author, tag, mapped_l_id,
                    new_u_id, text_val, u.get("行内位置", ""),
                    u.get("词性", ""), u.get("成分类型", ""), u.get("是否意象", ""),
                    u.get("大类编码", ""), u.get("子类编码", ""), u.get("感知通道", ""),
                    u.get("素材类型", ""), u.get("内部结构", ""), u.get("指涉来源", ""),
                    u.get("表现功能", ""), u.get("文化流通性", ""), u.get("跨文化性", ""),
                    u.get("认知强度", ""), u.get("核心意象", ""), u.get("结构功能组", ""),
                    u.get("情感极性", ""), u.get("情感类别", ""), u.get("情感置信度", "")
                ])
            # 3.3 解析表 B：情感轨迹
            for traj in p.get("情感轨迹", []):
                old_t_line = traj.get("诗行编号", "")
                mapped_t_line = line_id_map.get(old_t_line, old_t_line)
                rows_b.append(base_meta + [
                    mapped_t_line, line_text_map.get(mapped_t_line, ""),
                    traj.get("平均情感极性", ""), traj.get("主导情感", ""), traj.get("情感波动值", "")
                ])
            # 3.4 解析表 C：意象关系网络 (核心：修复指针)
            for r in p.get("意象关系", []):
                new_r_id = f"R{global_rel_idx:04d}"
                global_rel_idx += 1
                old_src = r.get("来源单元", "")
                old_tgt = r.get("目标单元", "")
                # 指针映射到新的全局 ID
                new_src = unit_id_map.get(old_src, old_src)
                new_tgt = unit_id_map.get(old_tgt, old_tgt)
                rows_c.append([new_p_id, title, author] + [
                    new_r_id,
                    new_src, unit_text_map.get(new_src, "未知"),
                    new_tgt, unit_text_map.get(new_tgt, "未知"),
                    r.get("关系类型", ""), r.get("关系强度", "")
                ])
    # --- 4. 写入 CSV 文件的标准函数 ---
    def save_csv(filename, header, rows):
        output_path = os.path.join(base_dir, filename)
        # utf-8-sig 防止 Excel 打开中文乱码
        with open(output_path, 'w', encoding='utf-8-sig', newline='') as f:
            writer = csv.writer(f)
            writer.writerow(header)
            writer.writerows(rows)
        return output_path
    if rows_a:
        path_a = save_csv("Table_A_Atomic.csv", header_a, rows_a)
        path_b = save_csv("Table_B_Lines.csv", header_b, rows_b)
        path_c = save_csv("Table_C_Relations.csv", header_c, rows_c)
        print("\n 数据转换与全局重排成功！文件已生成至当前目录：")
        print(f" Table A (共 {len(rows_a)} 个分析单元): {path_a}")
        print(f" Table B (共 {len(rows_b)} 条诗行轨迹): {path_b}")
        print(f" Table C (共 {len(rows_c)} 对意象关系): {path_c}")
    else:
        print("\n 警告：没有从 JSON 文件中提取到任何有效数据。")
if __name__ == "__main__":
    aggregate_jsons_to_csv()
```

-- 文件结束：json_to_3csv.py --

文件：split_1kb.py (55 行, 2.3 KB)

```
import os
def split_poetry_1kb_extreme(input_file, max_bytes=1024):
    if not os.path.exists(input_file):
        print(f" 找不到文件: {input_file}")
        return
```

```
    with open(input_file, 'r', encoding='utf-8') as f:
        content = f.read()
    poems = [p.strip() for p in content.split('==============================') if p.strip()]
    output_dir = "api_chunks_1kb"
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)
    print(f" 共读取到 {len(poems)} 首诗歌。启动【1KB 极限防爆防截断】切分引擎...")
    current_chunk = []
    current_bytes = 0
    file_index = 1
    for poem in poems:
        poem_text = f"{poem}\n\n==============================\n\n"
        # 核心：计算真实的 UTF-8 字节数 (中文字符占 3 字节)
        poem_bytes = len(poem_text.encode('utf-8'))
        # 如果加入这首诗会超过 1KB (1024 bytes)，且当前块里已经有数据了，立刻封包！
        if current_bytes + poem_bytes > max_bytes and current_chunk:
            output_path = os.path.join(output_dir, f"chunk_{file_index:03d}.txt")
            with open(output_path, 'w', encoding='utf-8') as out_f:
                out_f.write("".join(current_chunk))
            print(f" 生成切片: chunk_{file_index:03d}.txt (大小: {current_bytes} 字节)")
            # 重置状态，准备下一个文件
            file_index += 1
            current_chunk = []
            current_bytes = 0
        current_chunk.append(poem_text)
        current_bytes += poem_bytes
    # 把最后剩下的一点点尾料封包
    if current_chunk:
        output_path = os.path.join(output_dir, f"chunk_{file_index:03d}.txt")
        with open(output_path, 'w', encoding='utf-8') as out_f:
            out_f.write("".join(current_chunk))
        print(f" 生成收尾切片: chunk_{file_index:03d}.txt (大小: {current_bytes} 字节)")
    print(f" 全部切分完成！文件已安全存放在目录: {os.path.abspath(output_dir)}")
    print(f" 警告：拿着这些 1KB 的碎片去请求 API，绝对不会再触发截断！")
if __name__ == "__main__":
    # 确保你的古诗源文件名为 source.txt
    split_poetry_1kb_extreme('source.txt', max_bytes=1024)
-- 文件结束：split_1kb.py --
文件：tests/test_config.py  (63 行, 1.4 KB)
# -*- coding: utf-8 -*-
"""测试配置模块"""
import os
import sys
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
import config
def test_base_dir():
    assert os.path.isdir(config.BASE_DIR)
def test_api_urls():
    assert config.EMBED_API_URL.startswith("http")
    assert config.CHAT_API_URL.startswith("http")
def test_models():
    assert config.EMBED_MODEL
    assert config.CHAT_MODEL
def test_directories():
    assert config.POEM_JSON_DIR
    assert config.RAG_DB_DIR
    assert config.TEMPLATES_DIR
def test_category_map():
    assert len(config.CATEGORY_NAME_MAP) == 11
    assert config.CATEGORY_NAME_MAP["1-1"] == "自然意象-天文意象"
    assert config.CATEGORY_NAME_MAP["3-4"] == "人文意象-文化意象"
def test_major_category_map():
    assert config.CATEGORY_MAJOR_MAP["1"] == "自然意象"
    assert config.CATEGORY_MAJOR_MAP["2"] == "社会意象"
```

```
    assert config.CATEGORY_MAJOR_MAP["3"] == "人文意象"
def test_known_authors():
    assert "杜甫" in config.KNOWN_AUTHORS
    assert "李白" in config.KNOWN_AUTHORS
    assert len(config.KNOWN_AUTHORS) >= 20
def test_flask_config():
    assert config.FLASK_PORT == 5000
    assert config.FLASK_HOST == "0.0.0.0"
    assert config.SECRET_KEY
def test_pagination():
    assert config.PAGE_SIZE > 0
    assert config.RENDER_LIMIT > 0
def test_cache_config():
    assert config.CACHE_DEFAULT_TTL > 0
    assert isinstance(config.CACHE_ENABLED, bool)
```

-- 文件结束：tests/test_config.py --

文件：tests/test_validators.py （119 行，3.4 KB）

```
# -*- coding: utf-8 -*-
"""测试校验模块"""
import os
import sys
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
import pytest
from validators import (
    validate_question,
    validate_item,
    validate_history,
    validate_poem_id,
    validate_filename,
    sanitize_html,
)
from errors import ValidationError
class TestValidateQuestion:
    def test_valid_question(self):
        assert validate_question("杜甫写过哪些诗？") == "杜甫写过哪些诗？"
    def test_empty_question(self):
        with pytest.raises(ValidationError):
            validate_question("")
        with pytest.raises(ValidationError):
            validate_question(None)
        with pytest.raises(ValidationError):
            validate_question("   ")
    def test_question_stripped(self):
        assert validate_question("  秋天   ") == "秋天"
class TestValidateItem:
    def test_valid_item_with_text(self):
        item = {"文本": "明月", "line": "床前明月光"}
        assert validate_item(item) == item
    def test_valid_item_with_txt(self):
        item = {"txt": "青山", "line": "青山一道同云雨"}
        assert validate_item(item) == item
    def test_invalid_item(self):
        with pytest.raises(ValidationError):
            validate_item({})
        with pytest.raises(ValidationError):
            validate_item(None)
        with pytest.raises(ValidationError):
            validate_item("not a dict")
class TestValidateHistory:
    def test_empty_history(self):
        assert validate_history(None) == []
        assert validate_history([]) == []
    def test_valid_history(self):
        history = [
```

```
            {"role": "user", "content": "你好"},
            {"role": "assistant", "content": "你好！"},
        ]
        result = validate_history(history)
        assert len(result) == 2
    def test_filters_invalid_roles(self):
        history = [
            {"role": "user", "content": "你好"},
            {"role": "system", "content": "system msg"},
            {"role": "invalid", "content": "msg"},
        ]
        result = validate_history(history)
        assert len(result) == 1
    def test_filters_empty_content(self):
        history = [
            {"role": "user", "content": ""},
            {"role": "assistant", "content": None},
        ]
        result = validate_history(history)
        assert len(result) == 0
class TestValidatePoemId:
    def test_valid_ids(self):
        assert validate_poem_id("P001") == "P001"
        assert validate_poem_id("12345") == "12345"
    def test_invalid_ids(self):
        with pytest.raises(ValidationError):
            validate_poem_id("")
        with pytest.raises(ValidationError):
            validate_poem_id(None)
class TestValidateFilename:
    def test_default_name(self):
        assert validate_filename(None) == "export"
        assert validate_filename("") == "export"
    def test_valid_name(self):
        assert validate_filename("output.csv") == "output.csv"
        assert validate_filename("data.json") == "data.json"
    def test_invalid_name(self):
        with pytest.raises(ValidationError):
            validate_filename("bad.txt", allowed_exts=[".csv", ".json"])
class TestSanitizeHtml:
    def test_sanitize(self):
        assert sanitize_html("<script>") == "&lt;script&gt;"
        assert sanitize_html("a&b") == "a&amp;b"
    def test_empty(self):
        assert sanitize_html("") == ""
        assert sanitize_html(None) == ""
```

-- 文件结束：tests/test_validators.py --

文件：tests/test_analytics.py (161 行, 6.2 KB)

```
# -*- coding: utf-8 -*-
"""测试统计分析服务"""
import os
import sys
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from analytics import StatsService
SAMPLE_DATA = [
    {"poem_id": "P001", "title": "静夜思", "author": "李白",
     "txt": "明月", "文本": "明月", "cat": "自然意象-天文意象",
     "genre": "五言绝句", "emo_cat": "思乡", "情感类别": "思乡",
     "情感极性": "负面", "大类编码": "1", "子类编码": "1-1",
     "感知通道": "视觉"},
    {"poem_id": "P001", "title": "静夜思", "author": "李白",
     "txt": "月光", "文本": "月光", "cat": "自然意象-天文意象",
     "genre": "五言绝句", "emo_cat": "思乡", "情感类别": "思乡",
     "情感极性": "负面", "大类编码": "1", "子类编码": "1-1",
```

```
     "感知通道": "视觉"},
    {"poem_id": "P002", "title": "春晓", "author": "孟浩然",
     "txt": "春鸟", "文本": "春鸟", "cat": "自然意象-动物意象",
     "genre": "五言绝句", "emo_cat": "闲适", "情感类别": "闲适",
     "情感极性": "正面", "大类编码": "1", "子类编码": "1-4",
     "感知通道": "听觉"},
    {"poem_id": "P002", "title": "春晓", "author": "孟浩然",
     "txt": "风雨", "文本": "风雨", "cat": "自然意象-天文意象",
     "genre": "五言绝句", "emo_cat": "闲适", "情感类别": "闲适",
     "情感极性": "中性", "大类编码": "1", "子类编码": "1-1",
     "感知通道": "听觉"},
    {"poem_id": "P003", "title": "出塞", "author": "王昌龄",
     "txt": "明月", "文本": "明月", "cat": "自然意象-天文意象",
     "genre": "七言绝句", "emo_cat": "悲壮", "情感类别": "悲壮",
     "情感极性": "负面", "大类编码": "1", "子类编码": "1-1",
     "感知通道": "视觉"},
]
class TestStatsService:
    def setup_method(self):
        self.service = StatsService(SAMPLE_DATA)
    def test_total_images(self):
        assert self.service.total_images() == 5
    def test_total_poems(self):
        assert self.service.total_poems() == 3
    def test_total_authors(self):
        assert self.service.total_authors() == 3
    def test_top_images(self):
        top = self.service.top_images(5)
        assert len(top) > 0
        assert top[0]["text"] == "明月"
        assert top[0]["count"] == 2
    def test_category_distribution(self):
        cats = self.service.category_distribution()
        assert len(cats) == 2
        cat_map = {c["category"]: c["count"] for c in cats}
        assert cat_map["自然意象-天文意象"] == 4
        assert cat_map["自然意象-动物意象"] == 1
    def test_major_category_distribution(self):
        majors = self.service.major_category_distribution()
        assert len(majors) == 1
        assert majors[0]["category"] == "自然意象"
        assert majors[0]["count"] == 5
    def test_emotion_distribution(self):
        emos = self.service.emotion_distribution()
        emo_map = {e["emotion"]: e["count"] for e in emos}
        assert emo_map["思乡"] == 2
        assert emo_map["闲适"] == 2
        assert emo_map["悲壮"] == 1
    def test_emotion_polarity_distribution(self):
        pols = self.service.emotion_polarity_distribution()
        pol_map = {p["polarity"]: p["count"] for p in pols}
        assert pol_map["负面"] == 3
        assert pol_map["正面"] == 1
        assert pol_map["中性"] == 1
    def test_perception_channel_distribution(self):
        chs = self.service.perception_channel_distribution()
        ch_map = {c["channel"]: c["count"] for c in chs}
        assert ch_map["视觉"] == 3
        assert ch_map["听觉"] == 2
    def test_summary_report(self):
        report = self.service.summary_report()
        assert report["total_images"] == 5
        assert report["total_poems"] == 3
        assert report["total_authors"] == 3
```

```
        assert len(report["top_images"]) > 0
        assert len(report["category_stats"]) > 0
    def test_author_statistics(self):
        authors = self.service.author_statistics()
        author_map = {a["author"]: a for a in authors}
        assert "李白" in author_map
        assert author_map["李白"]["image_count"] == 2
    def test_cross_analysis(self):
        cross = self.service.cross_analysis()
        assert "自然意象-天文意象" in cross
        assert len(cross["自然意象-天文意象"]) > 0
    def test_empty_data(self):
        empty_service = StatsService([])
        assert empty_service.total_images() == 0
        assert empty_service.total_poems() == 0
        assert empty_service.top_images(10) == []
        assert empty_service.category_distribution() == []
        assert empty_service.emotion_distribution() == []
    def test_set_data(self):
        new_service = StatsService()
        assert new_service.total_images() == 0
        new_service.set_data(SAMPLE_DATA[:1])
        assert new_service.total_images() == 1
    def test_genre_distribution(self):
        genres = self.service.genre_distribution()
        genre_map = {g["genre"]: g["count"] for g in genres}
        assert genre_map["五言绝句"] == 4
        assert genre_map["七言绝句"] == 1
    def test_internal_structure_distribution(self):
        stats = self.service.internal_structure_distribution()
        assert len(stats) > 0
        all_structures = [s["structure"] for s in stats]
        assert "未标注" in all_structures
    def test_function_distribution(self):
        stats = self.service.function_distribution()
        assert len(stats) > 0
    def test_core_image_distribution(self):
        stats = self.service.core_image_distribution()
        assert len(stats) > 0
        assert stats[0]["core_image"] == "未标注"
    def test_reference_source_distribution(self):
        stats = self.service.reference_source_distribution()
        assert len(stats) > 0
    def test_cognitive_intensity_distribution(self):
        stats = self.service.cognitive_intensity_distribution()
        assert len(stats) > 0
    def test_material_type_distribution(self):
        stats = self.service.material_type_distribution()
        assert len(stats) > 0
    def test_cross_cultural_distribution(self):
        stats = self.service.cross_cultural_distribution()
        assert len(stats) > 0
-- 文件结束：tests/test_analytics.py --
文件：tests/test_cache.py  (115 行, 3.2 KB)
# -*- coding: utf-8 -*-
"""测试缓存模块"""
import os
import sys
import time
import tempfile
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
import pytest
from cache import MemoryCache, FileCache
class TestMemoryCache:
```

```
    def setup_method(self):
        self.cache = MemoryCache(default_ttl=60)
    def test_set_and_get(self):
        self.cache.set("key1", "value1")
        assert self.cache.get("key1") == "value1"
    def test_get_nonexistent(self):
        assert self.cache.get("nonexistent") is None
    def test_delete(self):
        self.cache.set("key_del", "to_delete")
        self.cache.delete("key_del")
        assert self.cache.get("key_del") is None
    def test_clear(self):
        self.cache.set("a", 1)
        self.cache.set("b", 2)
        self.cache.clear()
        assert self.cache.size() == 0
    def test_ttl_expiry(self):
        cache = MemoryCache(default_ttl=1)
        cache.set("expire_key", "will_expire")
        assert cache.get("expire_key") == "will_expire"
        time.sleep(1.5)
        assert cache.get("expire_key") is None
    def test_custom_ttl(self):
        self.cache.set("short", "quick", ttl=1)
        assert self.cache.get("short") == "quick"
        time.sleep(1.5)
        assert self.cache.get("short") is None
    def test_no_ttl(self):
        cache = MemoryCache(default_ttl=0)
        cache.set("no_ttl", "forever")
        assert cache.get("no_ttl") == "forever"
    def test_size(self):
        self.cache.set("s1", 1)
        self.cache.set("s2", 2)
        self.cache.set("s3", 3)
        assert self.cache.size() == 3
    def test_keys(self):
        self.cache.set("k1", 1)
        self.cache.set("k2", 2)
        keys = self.cache.keys()
        assert "k1" in keys
        assert "k2" in keys
    def test_complex_objects(self):
        obj = {"a": [1, 2, 3], "b": {"nested": "value"}}
        self.cache.set("complex", obj)
        assert self.cache.get("complex") == obj
class TestFileCache:
    def setup_method(self):
        self.tmpdir = tempfile.mkdtemp()
        self.cache = FileCache(cache_dir=self.tmpdir, default_ttl=60)
    def teardown_method(self):
        import shutil
        shutil.rmtree(self.tmpdir, ignore_errors=True)
    def test_set_and_get(self):
        self.cache.set("fkey", "fvalue")
        assert self.cache.get("fkey") == "fvalue"
    def test_get_nonexistent(self):
        assert self.cache.get("no_such_key") is None
    def test_delete(self):
        self.cache.set("del_key", "delete_me")
        self.cache.delete("del_key")
        assert self.cache.get("del_key") is None
    def test_clear(self):
        self.cache.set("a", 1)
```

```
        self.cache.set("b", 2)
        self.cache.clear()
        assert self.cache.size() == 0
    def test_special_chars_in_key(self):
        self.cache.set("path/to/file", "content")
        assert self.cache.get("path/to/file") == "content"
    def test_dict_value(self):
        data = {"name": "test", "values": [1, 2, 3]}
        self.cache.set("dict_key", data)
        assert self.cache.get("dict_key") == data
    def test_ttl_expiry(self):
        cache = FileCache(cache_dir=self.tmpdir, default_ttl=1)
        cache.set("expire", "gone")
        time.sleep(1.5)
        assert cache.get("expire") is None
-- 文件结束：tests/test_cache.py --
文件：tests/test_export.py  (89 行, 3.0 KB)
# -*- coding: utf-8 -*-
"""测试导出服务"""
import os
import sys
import tempfile
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
import pytest
# 重定向 EXPORT_DIR 到临时目录
import export_service
import config
from export_service import ExportService
SAMPLE_DATA = [
    {"poem_id": "P001", "title": "静夜思", "txt": "明月",
     "cat": "自然意象-天文意象", "genre": "五言绝句",
     "emotion": "思乡(负面)", "line": "床前明月光"},
    {"poem_id": "P002", "title": "春晓", "txt": "春鸟",
     "cat": "自然意象-动物意象", "genre": "五言绝句",
     "emotion": "闲适(正面)", "line": "处处闻啼鸟"},
]
class TestExportService:
    def setup_method(self):
        self.tmpdir = tempfile.mkdtemp()
        self.original_export_dir = config.EXPORT_DIR
        config.EXPORT_DIR = self.tmpdir
        # 重新加载模块级配置
        import importlib
        importlib.reload(export_service)
        # 重新导入以使用新配置
        from export_service import ExportService
        self.service = ExportService
    def teardown_method(self):
        import shutil
        shutil.rmtree(self.tmpdir, ignore_errors=True)
        config.EXPORT_DIR = self.original_export_dir
    def test_export_csv(self):
        path = self.service.export_traceback_to_csv(SAMPLE_DATA, "test.csv")
        assert os.path.exists(path)
        with open(path, "r", encoding="utf-8-sig") as f:
            content = f.read()
        assert "poem_id" in content
        assert "P001" in content
        assert "静夜思" in content
    def test_export_json(self):
        path = self.service.export_to_json(SAMPLE_DATA, "test.json")
        assert os.path.exists(path)
        with open(path, "r", encoding="utf-8") as f:
            data = f.read()
```

```
        assert "P001" in data
        assert "静夜思" in data
    def test_export_summary_report(self):
        report = {"total": 100, "items": ["a", "b"]}
        path = self.service.export_summary_report(report, "report.json")
        assert os.path.exists(path)
    def test_export_filtered_slice(self):
        path = self.service.export_filtered_slice(SAMPLE_DATA, 0, 1, "slice.json")
        assert os.path.exists(path)
        with open(path, "r", encoding="utf-8") as f:
            import json
            data = json.load(f)
        assert data["count"] == 1
        assert data["total"] == 2
    def test_list_exports(self):
        self.service.export_traceback_to_csv(SAMPLE_DATA, "list_test.csv")
        files = self.service.list_exports()
        assert len(files) >= 1
        assert any(f["name"] == "list_test.csv" for f in files)
    def test_clear_exports(self):
        self.service.export_traceback_to_csv(SAMPLE_DATA, "clear_me.csv")
        count = self.service.clear_exports()
        assert count >= 1
        assert len(self.service.list_exports()) == 0
    def test_export_empty_data_raises(self):
        with pytest.raises(Exception):
            self.service.export_traceback_to_csv([], "empty.csv")
```

-- 文件结束：tests/test_export.py --

文件：tests/test_models.py (109 行, 3.5 KB)

```
# -*- coding: utf-8 -*-
"""测试数据模型"""
import os
import sys
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from models import Poem, PoemLine, AnalysisUnit, EmotionTrajectory, ImageRelation
class TestPoemLine:
    def test_defaults(self):
        line = PoemLine()
        assert line.诗行编号 == ""
        assert line.原文 == ""
    def test_to_dict(self):
        line = PoemLine(诗行编号="L01", 原文="床前明月光")
        d = line.to_dict()
        assert d["诗行编号"] == "L01"
        assert d["原文"] == "床前明月光"
class TestAnalysisUnit:
    def test_is_image_true(self):
        unit = AnalysisUnit(是否意象="1")
        assert unit.is_image is True
    def test_is_image_false(self):
        unit = AnalysisUnit(是否意象="0")
        assert unit.is_image is False
    def test_to_dict(self):
        unit = AnalysisUnit(文本="明月", 词性="名词")
        d = unit.to_dict()
        assert d["文本"] == "明月"
        assert d["词性"] == "名词"
class TestPoem:
    def test_empty_poem(self):
        poem = Poem()
        assert poem.诗歌编号 == ""
        assert poem.image_count == 0
    def test_image_units_filter(self):
        poem = Poem(
```

```
            诗歌编号="P001",
            标题="静夜思",
            作者="李白",
            分析单元=[
                AnalysisUnit(文本="明月", 是否意象="1"),
                AnalysisUnit(文本="床", 是否意象="0"),
                AnalysisUnit(文本="霜", 是否意象="1"),
            ],
        )
        assert poem.image_count == 2
        assert len(poem.image_units) == 2
        assert poem.image_units[0].文本 == "明月"
    def test_summary(self):
        poem = Poem(标题="静夜思", 作者="李白", 诗歌编号="P001",
                    分析单元=[AnalysisUnit(是否意象="1")])
        assert "静夜思" in poem.summary
        assert "李白" in poem.summary
        assert "1个意象" in poem.summary
    def test_from_dict_full(self):
        data = {
            "诗歌编号": "P001",
            "标题": "静夜思",
            "作者": "李白",
            "分类标签": "五言绝句",
            "原文": "床前明月光",
            "诗行": [{"诗行编号": "L01", "原文": "床前明月光"}],
            "分析单元": [{"单元编号": "U001", "文本": "明月", "是否意象": "1"}],
            "情感轨迹": [{"诗行编号": "L01", "主导情感": "思乡"}],
            "意象关系": [{"关系类型": "并列", "来源文本": "明月", "目标文本": "床"}],
        }
        poem = Poem.from_dict(data)
        assert poem.诗歌编号 == "P001"
        assert poem.标题 == "静夜思"
        assert len(poem.诗行) == 1
        assert len(poem.分析单元) == 1
        assert len(poem.情感轨迹) == 1
        assert len(poem.意象关系) == 1
    def test_to_dict(self):
        poem = Poem(诗歌编号="P001", 标题="静夜思")
        d = poem.to_dict()
        assert d["诗歌编号"] == "P001"
        assert d["标题"] == "静夜思"
class TestEmotionTrajectory:
    def test_to_dict(self):
        t = EmotionTrajectory(诗行编号="L01", 主导情感="思乡")
        d = t.to_dict()
        assert d["诗行编号"] == "L01"
        assert d["主导情感"] == "思乡"
class TestImageRelation:
    def test_to_dict(self):
        r = ImageRelation(关系类型="并列", 关系强度="强")
        d = r.to_dict()
        assert d["关系类型"] == "并列"
        assert d["关系强度"] == "强"
```

-- 文件结束：tests/test_models.py --

文件：tests/test_preprocessor.py （124 行，4.1 KB）

```
# -*- coding: utf-8 -*-
"""测试数据预处理模块"""
import os
import sys
import json
import tempfile
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from preprocessor import (
```

```
    clean_json_content,
    validate_poem_json,
    parse_json_file,
    extract_image_statistics,
)
from models import Poem, AnalysisUnit
class TestCleanJsonContent:
    def test_remove_code_block(self):
        content = "```json\n{\"key\": \"value\"}\n```"
        result = clean_json_content(content)
        assert result == '{"key": "value"}'
    def test_no_block(self):
        content = '{"key": "value"}'
        result = clean_json_content(content)
        assert result == content
    def test_trailing_commas(self):
        content = '{"key": "value",}'
        result = clean_json_content(content)
        assert "value" in result
        assert result.endswith("}")
    def test_strip(self):
        content = "  \n  {'key': 'val'}  \n  "
        result = clean_json_content(content)
        assert result == "{'key': 'val'}"
class TestValidatePoemJson:
    def test_valid_basic(self):
        data = {"诗歌集": [{"标题": "静夜思", "诗歌编号": "001", "原文": "床前明月光"}]}
        errors = validate_poem_json(data)
        assert len(errors) == 0
    def test_missing_title(self):
        data = {"诗歌集": [{"诗歌编号": "001"}]}
        errors = validate_poem_json(data)
        assert any("标题" in e for e in errors)
    def test_missing_id(self):
        data = {"诗歌集": [{"标题": "静夜思"}]}
        errors = validate_poem_json(data)
        assert any("编号" in e for e in errors)
    def test_analysis_unit_validation(self):
        data = {
            "诗歌集": [{
                "标题": "静夜思",
                "诗歌编号": "001",
                "原文": "床前明月光",
                "诗行": [{"诗行编号": "1", "原文": "床前明月光"}],
                "分析单元": [
                    {"文本": "明月", "诗行编号": "1"},
                    {"诗行编号": "2"},  # missing text
                ],
            }]
        }
        errors = validate_poem_json(data)
        assert len(errors) == 1
        assert "分析单元" in errors[0]
        assert "文本" in errors[0]
class TestParseJsonFile:
    def test_valid_file(self):
        with tempfile.NamedTemporaryFile(mode="w", encoding="utf-8",
                                         suffix=".json", delete=False) as f:
            json.dump({"诗歌集": [{"标题": "静夜思", "诗歌编号": "001",
                                   "原文": "床前明月光"}]}, f, ensure_ascii=False)
            fpath = f.name
        data, errors = parse_json_file(fpath)
        assert data is not None
        assert len(errors) == 0
```

```
        os.unlink(fpath)
    def test_invalid_json(self):
        with tempfile.NamedTemporaryFile(mode="w", encoding="utf-8",
                                         suffix=".json", delete=False) as f:
            f.write("{bad json}")
            fpath = f.name
        data, errors = parse_json_file(fpath)
        assert data is None
        assert len(errors) > 0
        os.unlink(fpath)
    def test_file_not_found(self):
        data, errors = parse_json_file("/nonexistent/file.json")
        assert data is None
        assert any("读取" in e for e in errors)
class TestExtractImageStatistics:
    def test_basic_stats(self):
        poems = [
            Poem(标题="A", 作者="李白",
                 分析单元=[AnalysisUnit(文本="明月", 是否意象="1"),
                           AnalysisUnit(文本="床", 是否意象="0")]),
            Poem(标题="B", 作者="杜甫",
                 分析单元=[AnalysisUnit(文本="落日", 是否意象="1")]),
        ]
        stats = extract_image_statistics(poems)
        assert stats["total_poems"] == 2
        assert stats["total_images"] == 2
        assert stats["unique_images"] == 2
        assert stats["authors"] == 2
    def test_empty_poems(self):
        stats = extract_image_statistics([])
        assert stats["total_poems"] == 0
        assert stats["total_images"] == 0
```

-- 文件结束：tests/test_preprocessor.py --

文件：tests/test_middleware.py (91 行, 2.5 KB)

```
# -*- coding: utf-8 -*-
"""测试中间件模块"""
import os
import sys
import time
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from middleware import (
    RequestContext,
    PerformanceTracker,
    timer,
)
class TestRequestContext:
    def test_defaults(self):
        ctx = RequestContext()
        assert ctx.request_id is None
        assert ctx.start_time is None
    def test_elapsed(self):
        ctx = RequestContext()
        ctx.start_time = time.time() - 1.0
        elapsed = ctx.elapsed()
        assert elapsed >= 900
    def test_to_dict(self):
        ctx = RequestContext()
        ctx.request_id = "abc123"
        ctx.client_ip = "127.0.0.1"
        ctx.endpoint = "test"
        ctx.method = "GET"
        ctx.start_time = time.time()
        d = ctx.to_dict()
        assert d["request_id"] == "abc123"
```

```
        assert d["method"] == "GET"
class TestPerformanceTracker:
    def setup_method(self):
        self.tracker = PerformanceTracker()
    def test_record(self):
        self.tracker.record("/api/test", 100.0, 200)
        summary = self.tracker.summary()
        assert "/api/test" in summary
        assert summary["/api/test"]["calls"] == 1
        assert summary["/api/test"]["avg_ms"] == 100.0
    def test_multiple_records(self):
        for i in range(5):
            self.tracker.record("/api/test", float(i * 10), 200)
        summary = self.tracker.summary()
        assert summary["/api/test"]["calls"] == 5
        assert summary["/api/test"]["max_ms"] == 40.0
        assert summary["/api/test"]["min_ms"] == 0.0
    def test_error_counting(self):
        self.tracker.record("/api/test", 50.0, 200)
        self.tracker.record("/api/test", 50.0, 500)
        self.tracker.record("/api/test", 50.0, 400)
        summary = self.tracker.summary()
        assert summary["/api/test"]["errors"] == 2
        assert summary["/api/test"]["error_rate_pct"] == 66.7
    def test_reset(self):
        self.tracker.record("/api/test", 100.0, 200)
        self.tracker.reset()
        assert self.tracker.summary() == {}
    def test_multiple_endpoints(self):
        self.tracker.record("/api/a", 100.0, 200)
        self.tracker.record("/api/b", 200.0, 200)
        summary = self.tracker.summary()
        assert len(summary) == 2
class TestTimerDecorator:
    def test_timer_basic(self):
        @timer
        def my_func():
            return 42
        assert my_func() == 42
    def test_timer_with_args(self):
        @timer
        def add(a, b):
            return a + b
        assert add(3, 4) == 7
```

-- 文件结束：tests/test_middleware.py --

文件：tests/test_cli.py （32 行，0.7 KB）

```
# -*- coding: utf-8 -*-
"""测试 CLI 模块"""
import os
import sys
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from cli import cmd_status, cmd_scan, cmd_clear_cache, cmd_list_exports
class MockArgs:
    def __init__(self, **kwargs):
        for k, v in kwargs.items():
            setattr(self, k, v)
class TestCliCommands:
    def test_status_runs(self):
        """至少不抛出异常"""
        args = MockArgs()
        try:
            cmd_status(args)
        except SystemExit:
            pass
```

```
    def test_clear_cache_runs(self):
        args = MockArgs()
        try:
            cmd_clear_cache(args)
        except Exception:
            pass
-- 文件结束：tests/test_cli.py --
文件：tests/test_utils.py  (173 行, 4.1 KB)
# -*- coding: utf-8 -*-
"""测试通用工具函数"""
import os
import sys
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from utils import (
    truncate,
    is_chinese_char,
    chinese_ratio,
    split_sentences,
    format_file_size,
    safe_get,
    dict_pick,
    dict_omit,
    md5_hash,
    safe_json_loads,
    list_chunk,
    deduplicate_by_key,
    normalize_whitespace,
    extract_numbers,
    merge_dicts,
)
class TestTruncate:
    def test_short_string(self):
        assert truncate("hello") == "hello"
    def test_long_string(self):
        result = truncate("hello world", max_len=5)
        assert result == "hello..."
    def test_empty(self):
        assert truncate("") == ""
        assert truncate(None) == ""
class TestIsChineseChar:
    def test_chinese(self):
        assert is_chinese_char("中") is True
        assert is_chinese_char("国") is True
    def test_not_chinese(self):
        assert is_chinese_char("a") is False
        assert is_chinese_char("1") is False
        assert is_chinese_char("") is False
class TestChineseRatio:
    def test_all_chinese(self):
        assert chinese_ratio("中文") == 1.0
    def test_mixed(self):
        ratio = chinese_ratio("hello中文")
        assert abs(ratio - 0.2857) < 0.001
    def test_empty(self):
        assert chinese_ratio("") == 0
class TestSplitSentences:
    def test_basic(self):
        result = split_sentences("句一。句二！句三？")
        assert len(result) == 3
    def test_no_punctuation(self):
        assert split_sentences("一句话") == ["一句话"]
    def test_empty(self):
        assert split_sentences("") == []
class TestFormatFileSize:
```

```
    def test_bytes(self):
        assert "B" in format_file_size(500)
    def test_kb(self):
        assert "KB" in format_file_size(2048)
    def test_mb(self):
        assert "MB" in format_file_size(1048576)
    def test_zero(self):
        assert format_file_size(0) == "0 B"
class TestSafeGet:
    def test_existing_key(self):
        assert safe_get({"a": 1}, "a") == 1
    def test_missing_key(self):
        assert safe_get({"a": 1}, "b") == ""
    def test_custom_default(self):
        assert safe_get({"a": 1}, "b", default=None) is None
    def test_none_value(self):
        assert safe_get({"a": None}, "a") == ""
class TestDictPick:
    def test_basic(self):
        result = dict_pick({"a": 1, "b": 2, "c": 3}, ["a", "c"])
        assert result == {"a": 1, "c": 3}
class TestDictOmit:
    def test_basic(self):
        result = dict_omit({"a": 1, "b": 2, "c": 3}, ["b"])
        assert result == {"a": 1, "c": 3}
class TestMd5Hash:
    def test_basic(self):
        h = md5_hash("hello")
        assert len(h) == 32
        assert h == md5_hash("hello")
    def test_different(self):
        assert md5_hash("a") != md5_hash("b")
class TestSafeJsonLoads:
    def test_valid(self):
        result = safe_json_loads('{"a": 1}')
        assert result == {"a": 1}
    def test_invalid(self):
        result = safe_json_loads("{bad}", default={})
        assert result == {}
class TestListChunk:
    def test_basic(self):
        result = list_chunk([1, 2, 3, 4, 5], 2)
        assert result == [[1, 2], [3, 4], [5]]
    def test_empty(self):
        assert list_chunk([], 2) == []
class TestDeduplicateByKey:
    def test_basic(self):
        data = [{"id": 1, "v": "a"}, {"id": 2, "v": "b"}, {"id": 1, "v": "c"}]
        result = deduplicate_by_key(data, "id")
        assert len(result) == 2
        assert result[0]["v"] == "a"
class TestNormalizeWhitespace:
    def test_basic(self):
        assert normalize_whitespace("  a   b  c  ") == "a b c"
    def test_empty(self):
        assert normalize_whitespace("") == ""
        assert normalize_whitespace(None) == ""
class TestExtractNumbers:
    def test_basic(self):
        assert extract_numbers("abc123def456") == [123, 456]
    def test_no_numbers(self):
        assert extract_numbers("abc") == []
class TestMergeDicts:
    def test_basic(self):
```

```
  121          result = merge_dicts({"a": 1, "b": 2}, {"b": 3, "c": 4})
  122          assert result == {"a": 1, "b": 3, "c": 4}
-- 文件结束：tests/test_utils.py --
```